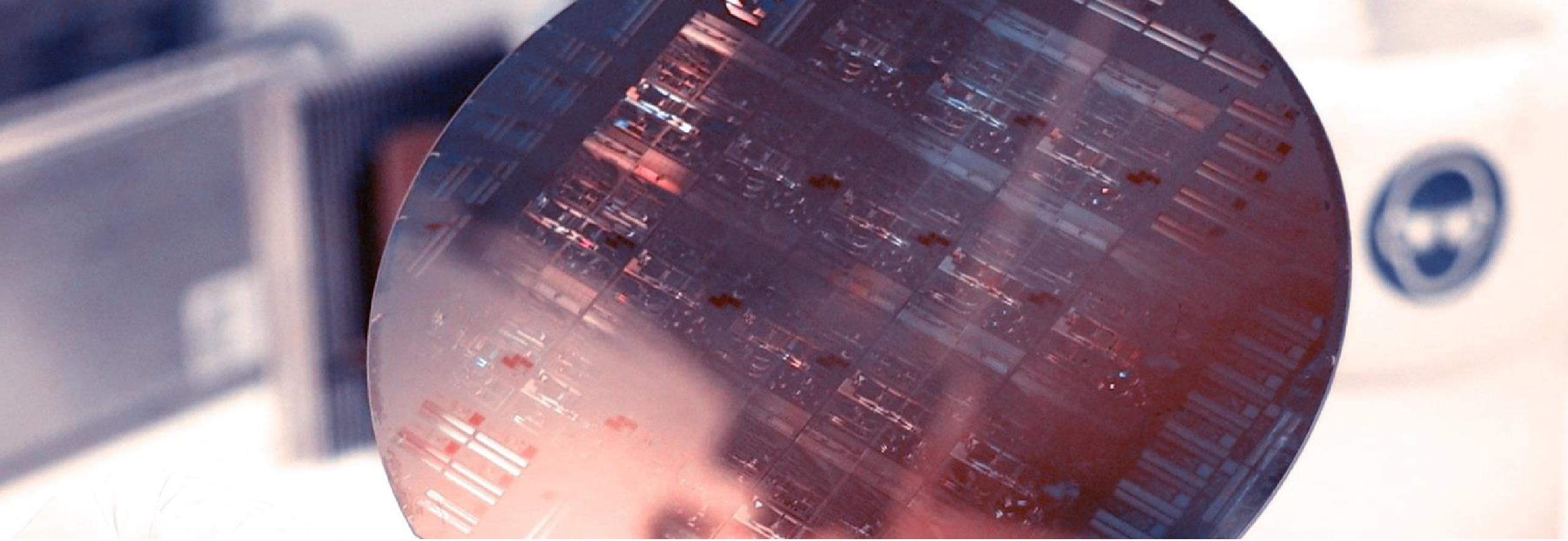


PIC-UPV
PERTE CHIP CHAIR

Lab Book 3.0 Circuits and Compact Models

Student: Claudia Quiles y Joan Enric Carcel

Financiado por la Unión Europea NextGenerationEU
GOBIERNO DE ESPAÑA MINISTERIO PARA LA TRANSFORMACIÓN DIGITAL Y DE LA FUNCIÓN PÚBLICA
Plan de Recuperación, Transformación y Resiliencia
España | digital 2026
PERTE Chip
UNIVERSITAT POLITÈCNICA DE VALÈNCIA
TSI-069100-2023-0002

Instructions

General

SCRIPTS

- From GitHub →
  - Fork the repo: https://github.com/pascualmunoz/cifoin-lab4.git
  - Clone the repo using VSCode
  - Open folder / repo in VSCode
  - Open terminal: uv sync
- Run the Jupyter Notebook:
  - LAYOUT_STUDENT.ipynb

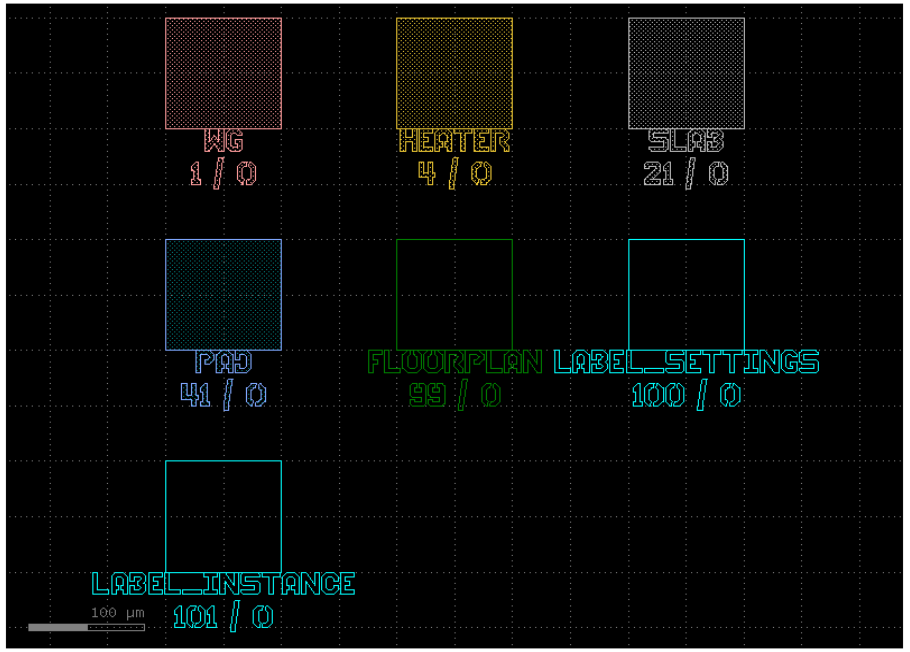
- **Points**
  - **LO1: 3 points**
  - **LO2: 3 points**
  - **LO3: 1 points**
  - **LO4: 3 points**

Financiado por la Unión Europea NextGenerationEU
GOBIERNO DE ESPAÑA MINISTERIO PARA LA TRANSFORMACIÓN DIGITAL Y DE LA FUNCIÓN PÚBLICA
Plan de Recuperación, Transformación y Resiliencia
España | digital 2026
PERTE Chip
UNIVERSITAT POLITÈCNICA DE VALÈNCIA
TSI-069100-2023-0002

Learning outcome #1a – Notebook section 1. Technology

“We will work with the demo UPVfab PDK. Let's view the layer distribution of this PDK”

```
LAYER_VIEWS = PDK.layer_views
c = LAYER_VIEWS.preview_layerset()
c.show()
c.plot()
```



Un PDK es el conjunto de herramientas, archivos de tecnología y componentes verificados que proporciona una fábrica (foundry) para diseñar chips compatibles con sus procesos de fabricación.
En general, las capas para el layout en un PDK Representan las diferentes máscaras litográficas y pasos físicos de fabricación (grabado de silicio, dopado, metalización, nitruro, etc.).
For our particular PDK we find the following layers, with number and label/description:

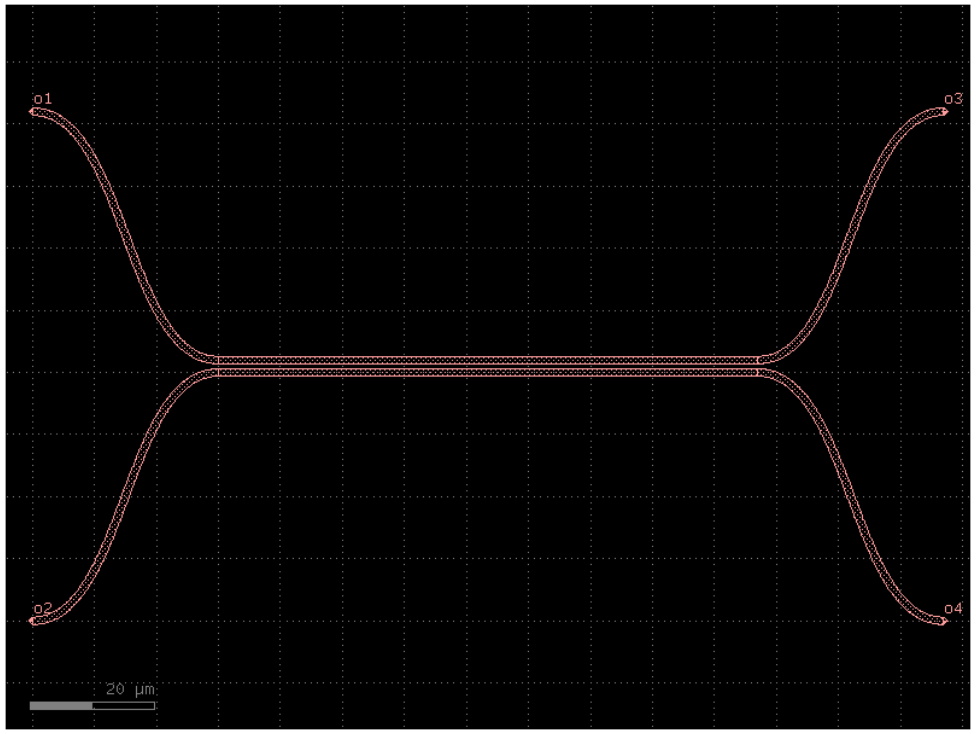
- WG (Layer 1/0): Guía de ondas (Waveguide) principal.
- HEATER (Layer 4/0): Capa metálica para los calentadores térmicos.
- SLAB (Layer 21/0): Capa para guías de onda tipo Rib (grabado parcial).
- PAD (Layer 41/0): Almohadillas metálicas para conexiones eléctricas externas.
- FLOORPLAN (Layer 99/0): Contorno físico o línea de corte del chip (Die).

Financiado por la Unión Europea NextGenerationEU
GOBIERNO DE ESPAÑA MINISTERIO PARA LA TRANSFORMACIÓN DIGITAL Y DE LA FUNCIÓN PÚBLICA
Plan de Recuperación, Transformación y Resiliencia
España | digital 2026
PERTE Chip
UNIVERSITAT POLITÈCNICA DE VALÈNCIA
TSI-069100-2023-0002

Learning outcome #1b – Notebook section 1. Technology – DC component

"Use your design results from Lab2 and Lab3 to create layout instances of your designed components: DCs, MMIs, MZIs & Ring Resonators".

```python
# Creamos el componente
c = gf.Component()

# Parametros
length: float = 86.982
gap: float = 0.8
dx: float = 30.0
dy: float = 40.0
width: float = 1.2

# Definimos la sección transversal tipo Strip con ancho de 1.2 micras en la capa 'WG'
xs = gf.cross_section.strip(width=1.2, layer="WG")

# Región de acoplo
region_central = c.add_ref(gf.components.coupler_straight( length=length, gap=gap,cross_section=xs))
#izquierda
s_bend_sup_izq = c.add_ref(gf.components.bend_s(size=(dx, dy), cross_section=xs))
s_bend_sup_izq.connect("o1", region_central.ports["o1"])
s_bend_inf_izq = c.add_ref(gf.components.bend_s( size=(dx, dy), cross_section=xs)).mirror()
s_bend_inf_izq.connect("o1", region_central.ports["o2"])
#derecha
s_bend_sup_der = c.add_ref( gf.components.bend_s(size=(dx, dy), cross_section=xs))
s_bend_sup_der.connect("o1", region_central.ports["o3"])
s_bend_inf_der = c.add_ref(gf.components.bend_s(size=(dx, dy), cross_section=xs)).mirror()
s_bend_inf_der.connect("o1", region_central.ports["o4"])

# Puertos
c.add_port(name="o1", port=s_bend_inf_izq.ports["o2"]) # Entrada Inferior Izquierda
c.add_port(name="o2", port=s_bend_sup_izq.ports["o2"]) # Entrada Superior Izquierda
c.add_port(name="o3", port=s_bend_sup_der.ports["o2"]) # Salida Superior Derecha
c.add_port(name="o4", port=s_bend_inf_der.ports["o2"]) # Salida Inferior Derecha

c.draw_ports()

c.plot()
```



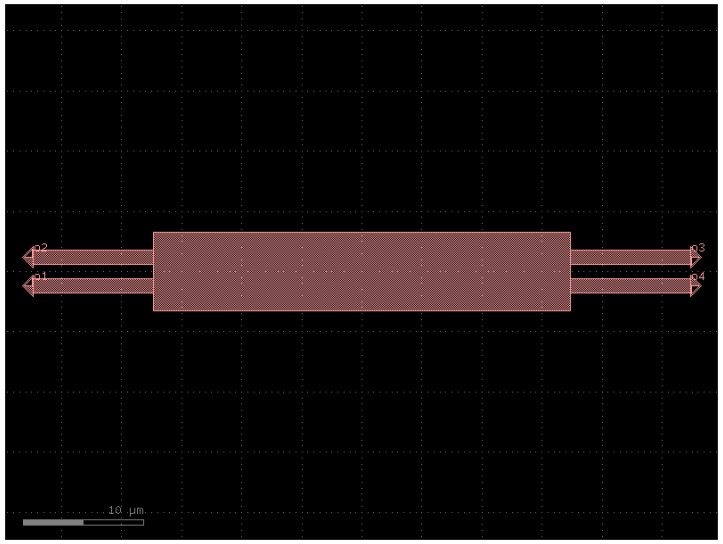
Este es el acoplador direccional (DC) diseñado en Lab2. Es un divisor 2x2 por interferencia de modos acoplados con constante de acoplamiento objetivo K = 0.5. Las dimensiones relevantes son: longitud de la región de acoplamiento length = 86.982 µm, gap entre guías = 0.8 µm y ancho de guía width = 1.2 µm. Las posiciones de las entradas/salidas respecto al centro están separadas ±40.0 µm en Y y ±30.0 µm en X gracias a las curvas en S.

Financiado por la Unión Europea NextGenerationEU
Gobierno de España · Ministerio para la Transformación Digital y de la Función Pública
Plan de Recuperación, Transformación y Resiliencia
España | digital 2026
PERTE Chip
Universitat Politècnica de València
TSI-069100-2023-0002

Learning outcome #1b – Notebook section 1. Technology – MMI component

"Use your design results from Lab2 and Lab3 to create layout instances of your designed components: DCs, MMIs, MZIs & Ring Resonators"

```python
gf.clear_cache()

c = gf.Component()
# Parametros
W_MMI   = float(6.6)      # Ancho del bloque multimodo (mmi_Width del Lab2)
L_MMI   = float(34.718)   # L_MMI + dL_MMI = 34.518 + 0.2 del Lab2
W_wg    = float(1.8)      # Ancho guías de acceso (IO wg width del Lab2)1+0.8
L_wg   = float(10.0)
dy      = float(0.1)      # Offset de posición de los puertos

# Posiciones de los puertos fórmula del Lab2:
pos_inf = W_MMI * (-1/6) + (-dy)   # Puerto inferior
pos_sup = W_MMI * ( 1/6) + ( dy)   # Puerto superior

# Bloque central
mmi_core = c.add_ref(gf.components.rectangle(size=(L_MMI, W_MMI), layer="WG"))
# Centrar el bloque en el origen
mmi_core.dmove((-L_MMI / 2, -W_MMI / 2))

# Guias de acceso
xs = gf.cross_section.strip(width = W_wg, layer = 'WG')
wg = gf.components.straight(length = L_wg, cross_section=xs_personalizado)

# Entrada superior izquierda (o2)
t_o2 = c.add_ref(wg)
t_o2.drotate(180)
t_o2.dmove((-L_MMI / 2, pos_sup))

# Entrada inferior izquierda (o1)
t_o1 = c.add_ref(wg)
t_o1.drotate(180)
t_o1.dmove((-L_MMI / 2, pos_inf))

# Salida superior derecha (o3)
t_o3 = c.add_ref(wg)
t_o3.dmove((L_MMI / 2, pos_sup))

# Salida inferior derecha (o4)
t_o4 = c.add_ref(wg)
t_o4.dmove((L_MMI / 2, pos_inf))

# Puertos externos
c.add_port(name="o1", center=(-L_MMI/2 - L_wg, pos_inf),width=W_wg, orientation=180, layer="WG")
c.add_port( name="o2", center=(-L_MMI/2 - L_wg, pos_sup),width=W_wg, orientation=180, layer="WG")
c.add_port(name="o3", center=( L_MMI/2 + L_wg, pos_sup),width=W_wg, orientation=0,   layer="WG")
c.add_port(name="o4", center=( L_MMI/2 + L_wg, pos_inf),width=W_wg, orientation=0,   layer="WG")

c.draw_ports()

c.plot()
```



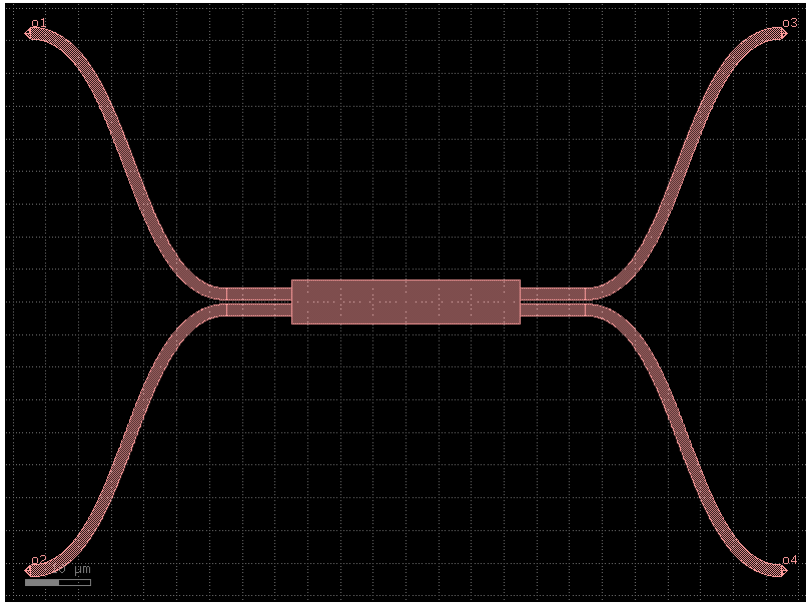
Esta es la MMI 2x2 diseñada en Lab2. Es un divisor por interferencia multimodo con constante de acoplamiento objetivo K = 0.5. Las dimensiones relevantes son: ancho del bloque W_MMI = 6.6 µm, longitud del bloque L_MMI = 34.718 µm (correspondiente a L_MMI + dL_MMI = 34.518 + 0.2 µm del Lab2) y ancho de las guías de acceso W_wg = 1.8 µm. Las posiciones de las entradas/salidas respecto al centro del bloque se calculan como ±W_MMI/6 ± dy, resultando en ±1.1 µm con un offset de ajuste dy = 0.1 µm, con guías de acceso de longitud L_wg = 10.0 µm a cada lado.

Financiado por la Unión Europea NextGenerationEU
GOBIERNO DE ESPAÑA MINISTERIO PARA LA TRANSFORMACIÓN DIGITAL Y DE LA FUNCIÓN PÚBLICA
Plan de Recuperación, Transformación y Resiliencia
España | digital 2026
PERTE Chip
UNIVERSITAT POLITÈCNICA DE VALÈNCIA
TSI-069100-2023-0002

Learning outcome #1b – Notebook section 1. Technology – MMI_bend component

"Use your design results from Lab2 and Lab3 to create layout instances of your designed components: DCs, MMIs, MZIs & Ring Resonators"

```python
gf.clear_cache()

c = gf.Component()
# Parametros
W_MMI   = float(6.6)      # Ancho del bloque multimodo (mmi_Width del Lab2)
L_MMI   = float(34.718)   # L_MMI + dL_MMI = 34.518 + 0.2 del Lab2
W_wg    = float(1.8)      # Ancho guías de acceso (IO wg width del Lab2)1+0.8
L_wg    = float(10.0)
dy      = float(0.1)      # Offset de posición de los puertos
dx_bend = float(30.0)
dy_bend = float(40.0)

# Posiciones de los puertos:
pos_inf = W_MMI * (-1/6) + (-dy)   # Puerto inferior
pos_sup = W_MMI * ( 1/6) + ( dy)   # Puerto superior

# Bloque central
mmi_core = c.add_ref(
    gf.components.rectangle(size=(L_MMI, W_MMI), layer="WG"))
# Centrar el bloque en el origen
mmi_core.dmove((-L_MMI / 2, -W_MMI / 2))

# Guias de acceso
xs = gf.cross_section.strip(width = W_wg, layer = 'WG')
wg = gf.components.straight(length = L_wg, cross_section=xs)

# Entrada superior izquierda (o2)
t_o2 = c.add_ref(wg)
t_o2.drotate(180)
t_o2.dmove((-L_MMI / 2, pos_sup))

# Entrada inferior izquierda (o1)
t_o1 = c.add_ref(wg)
t_o1.drotate(180)
t_o1.dmove((-L_MMI / 2, pos_inf))

# Salida superior derecha (o3)
t_o3 = c.add_ref(wg)
t_o3.dmove((L_MMI / 2, pos_sup))

# Salida inferior derecha (o4)
t_o4 = c.add_ref(wg)
t_o4.dmove((L_MMI / 2, pos_inf))


#  BEND_S conectados al final de cada guía
# Izquierda superior
bs_sup_izq = c.add_ref(gf.components.bend_s(size=(dx_bend, dy_bend), cross_section=xs))
bs_sup_izq.connect("o1", t_o1.ports["o2"])

# Izquierda inferior
bs_inf_izq = c.add_ref(gf.components.bend_s(size=(dx_bend, dy_bend), cross_section=xs)).mirror()
bs_inf_izq.connect("o1", t_o2.ports["o2"])

# Derecha superior
bs_sup_der = c.add_ref(gf.components.bend_s(size=(dx_bend, dy_bend), cross_section=xs))
bs_sup_der.connect("o1", t_o3.ports["o2"])

# Derecha inferior
bs_inf_der = c.add_ref(gf.components.bend_s(size=(dx_bend, dy_bend), cross_section=xs)).mirror()
bs_inf_der.connect("o1", t_o4.ports["o2"])

# Puertos al final de los bend_s
c.add_port(name="o1", port=bs_inf_izq.ports["o2"])  # Entrada inferior izq
c.add_port(name="o2", port=bs_sup_izq.ports["o2"])  # Entrada superior izq
c.add_port(name="o3", port=bs_sup_der.ports["o2"])  # Salida superior der
c.add_port(name="o4", port=bs_inf_der.ports["o2"])  # Salida inferior der
c.draw_ports()

c.plot()
```



Esta es la MMI 2x2 con curvas en S, versión extendida del bloque anterior. Mantiene las mismas dimensiones del núcleo multimodo: W_MMI = 6.6 µm, L_MMI = 34.718 µm y W_wg = 1.8 µm con guías de acceso de L_wg = 10.0 µm. La diferencia respecto a la versión anterior es que cada puerto lleva conectada una curva en S de dimensiones dx_bend = 30.0 µm en X y dy_bend = 40.0 µm en Y, lo que separa las entradas y salidas hasta ±40.0 µm respecto al eje central, facilitando la conexión con otros componentes del layout.

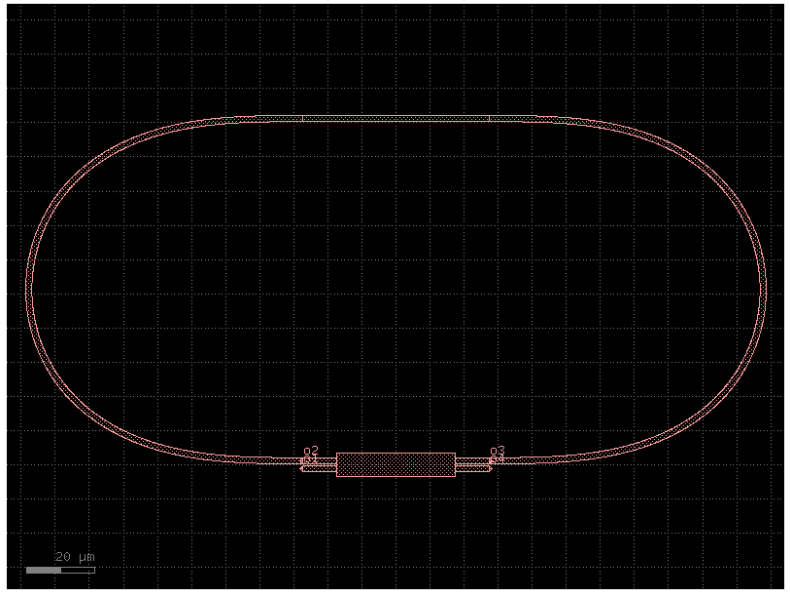
Financiado por la Unión Europea NextGenerationEU
Plan de Recuperación, Transformación y Resiliencia
España | digital 2026
PERTE Chip
UNIVERSITAT POLITÈCNICA DE VALÈNCIA
TSI-069100-2023-0002

Learning outcome #1b – Notebook section 1. Technology – Ring component

"Use your design results from Lab2 and Lab3 to create layout instances of your designed components: DCs, MMIs, MZIs & Ring Resonators"

```python
gf.clear_cache()

c  = gf.Component()
#parametros
W_MMI   = float(6.6)
L_MMI   = float(34.718)
W_wg    = float(1.8)
L_wg    = float(10.0)
dy      = float(0.1)
radio   = float(50.0)
xs = gf.cross_section.strip(width=W_wg, layer="WG")

# MMI central
# Posiciones de los puertos:
pos_inf = W_MMI * (-1/6) + (-dy)   # Puerto inferior
pos_sup = W_MMI * ( 1/6) + ( dy)   # Puerto superior

# Bloque central
mmi_core = c.add_ref(gf.components.rectangle(
    size=(L_MMI, W_MMI), layer="WG"))
# Centrar el bloque en el origen
mmi_core.dmove((-L_MMI / 2, -W_MMI / 2))

# Guias de acceso
wg = gf.components.straight(
    length = L_wg, cross_section=xs)

# Entrada superior izquierda (o2)
t_o2 = c.add_ref(wg)
t_o2.drotate(180)
t_o2.dmove((-L_MMI / 2, pos_sup))

# Entrada inferior izquierda (o1)
t_o1 = c.add_ref(wg)
t_o1.drotate(180)
t_o1.dmove((-L_MMI / 2, pos_inf))

# Salida superior derecha (o3)
t_o3 = c.add_ref(wg)
t_o3.dmove((L_MMI / 2, pos_sup))

# Salida inferior derecha (o4)
t_o4 = c.add_ref(wg)
t_o4.dmove((L_MMI / 2, pos_inf))
```

```python
#--------------------------------
# Lado derecho
bend_der_1 = c.add_ref(gf.components.bend_euler(angle=180, radius=radio, cross_section=xs))
bend_der_1.connect("o1", t_o3.ports["o2"])

# Lado izquierdo
bend_izq_1 = c.add_ref(gf.components.bend_euler(angle=-180, radius=radio, cross_section=xs))
bend_izq_1.connect("o1", t_o2.ports["o2"])

# Separación entre o3 y o2 x
sep_puertos = abs(t_o2.ports["o2"].dy - t_o3.ports["o2"].dy)

# Recto superior
recto = c.add_ref(gf.components.straight(length=sep_puertos , cross_section=xs))
recto.connect("o1", bend_der_1.ports["o2"])

gf.routing.route_single(component=c,port1=recto.ports["o2"],port2=bend_izq_1.ports["o2"],cross_section=xs)

# Puertos externos
c.add_port(name="o1", port=t_o1.ports["o2"])
c.add_port(name="o2", port=t_o2.ports["o2"])
c.add_port(name="o3", port=t_o3.ports["o2"])
c.add_port(name="o4", port=t_o4.ports["o2"])

c.draw_ports()
c.plot()
```



Este es el resonador de anillo 2x2 Mantiene el mismo núcleo multimodo con W_MMI = 6.6 µm, L_MMI = 34.718 µm y guías de acceso de W_wg = 1.8 µm y L_wg = 10.0 µm. El anillo se cierra mediante dos curvas de Euler de 180° con radio = 50.0 µm, una a cada lado de la MMI, conectadas por una guía recta superior de longitud igual a la separación vertical entre los puertos o2 y o3. Los puertos externos o1 y o4 actúan como entrada y salida del dispositivo.

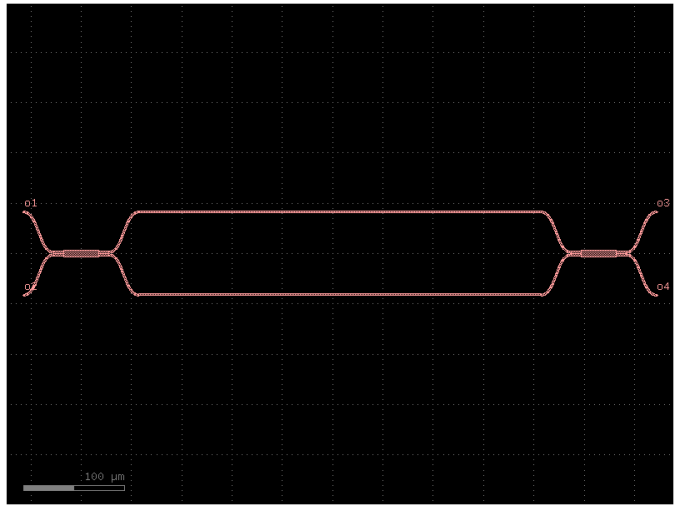
Financiado por la Unión Europea NextGenerationEU
Gobierno de España · Ministerio para la Transformación Digital y de la Función Pública
Plan de Recuperación, Transformación y Resiliencia
España | digital 2026
PERTE Chip
Universitat Politècnica de València
TSI-069100-2023-0002

Learning outcome #1b – Notebook section 1. Technology – MZI component

"Use your design results from Lab2 and Lab3 to create layout instances of your designed components: DCs, MMIs, MZIs & Ring Resonators"

```python
c = gf.Component()
xs = gf.cross_section.strip(width=1.8, layer="WG")
delta_length= float(0.0)
L_brazo=float(400.0)
# — 1. SPLITTER (DC izquierdo) ————————————————————
W_MMI=6.6; L_MMI=34.718; W_wg=1.8; L_wg=10.0; dy=0.1; dx_bend=30.0; dy_bend=40.0

splitter = gf.Component()
pos_inf = W_MMI * (-1/6) + (-dy)
pos_sup = W_MMI * ( 1/6) + ( dy)

mmi_core = splitter.add_ref(gf.components.rectangle(size=(L_MMI, W_MMI), layer="WG"))
mmi_core.dmove((-L_MMI / 2, -W_MMI / 2))

xs_mmi = gf.cross_section.strip(width=W_wg, layer='WG')
wg = gf.components.straight(length=L_wg, cross_section=xs_mmi)

t_o2 = splitter.add_ref(wg); t_o2.drotate(180); t_o2.dmove((-L_MMI/2, pos_sup))
t_o1 = splitter.add_ref(wg); t_o1.drotate(180); t_o1.dmove((-L_MMI/2, pos_inf))
t_o3 = splitter.add_ref(wg); t_o3.dmove((L_MMI/2, pos_sup))
t_o4 = splitter.add_ref(wg); t_o4.dmove((L_MMI/2, pos_inf))

bs_sup_izq = splitter.add_ref(gf.components.bend_s(size=(dx_bend, dy_bend), cross_section=xs_mmi))
bs_sup_izq.connect("o1", t_o1.ports["o2"])
bs_inf_izq = splitter.add_ref(gf.components.bend_s(size=(dx_bend, dy_bend), cross_section=xs_mmi)).mirror()
bs_inf_izq.connect("o1", t_o2.ports["o2"])
bs_sup_der = splitter.add_ref(gf.components.bend_s(size=(dx_bend, dy_bend), cross_section=xs_mmi))
bs_sup_der.connect("o1", t_o3.ports["o2"])
bs_inf_der = splitter.add_ref(gf.components.bend_s(size=(dx_bend, dy_bend), cross_section=xs_mmi)).mirror()
bs_inf_der.connect("o1", t_o4.ports["o2"])

splitter.add_port(name="o1", port=bs_inf_izq.ports["o2"])
splitter.add_port(name="o2", port=bs_sup_izq.ports["o2"])
splitter.add_port(name="o3", port=bs_sup_der.ports["o2"])
splitter.add_port(name="o4", port=bs_inf_der.ports["o2"])

splitter_ref = c.add_ref(splitter)

# — 2. BRAZOS ————————————————————————————————
brazo_sup = c.add_ref(gf.components.straight(length=L_brazo + delta_length, cross_section=xs))
brazo_inf = c.add_ref(gf.components.straight(length=L_brazo, cross_section=xs))

brazo_sup.connect("o1", splitter_ref.ports["o3"])
brazo_inf.connect("o1", splitter_ref.ports["o4"])
```

```python
# — 3. COMBINER (DC derecho) ——————————————————————
combiner = gf.Component()
pos_inf = W_MMI * (-1/6) + (-dy)
pos_sup = W_MMI * ( 1/6) + ( dy)

mmi_core2 = combiner.add_ref(gf.components.rectangle(size=(L_MMI, W_MMI), layer="WG"))
mmi_core2.dmove((-L_MMI / 2, -W_MMI / 2))

t_o2b = combiner.add_ref(wg); t_o2b.drotate(180); t_o2b.dmove((-L_MMI/2, pos_sup))
t_o1b = combiner.add_ref(wg); t_o1b.drotate(180); t_o1b.dmove((-L_MMI/2, pos_inf))
t_o3b = combiner.add_ref(wg); t_o3b.dmove((L_MMI/2, pos_sup))
t_o4b = combiner.add_ref(wg); t_o4b.dmove((L_MMI/2, pos_inf))

bs_sup_izq2 = combiner.add_ref(gf.components.bend_s(size=(dx_bend, dy_bend), cross_section=xs_mmi))
bs_sup_izq2.connect("o1", t_o1b.ports["o2"])
bs_inf_izq2 = combiner.add_ref(gf.components.bend_s(size=(dx_bend, dy_bend), cross_section=xs_mmi)).mirror()
bs_inf_izq2.connect("o1", t_o2b.ports["o2"])
bs_sup_der2 = combiner.add_ref(gf.components.bend_s(size=(dx_bend, dy_bend), cross_section=xs_mmi))
bs_sup_der2.connect("o1", t_o3b.ports["o2"])
bs_inf_der2 = combiner.add_ref(gf.components.bend_s(size=(dx_bend, dy_bend), cross_section=xs_mmi)).mirror()
bs_inf_der2.connect("o1", t_o4b.ports["o2"])

combiner.add_port(name="o1", port=bs_inf_izq2.ports["o2"])
combiner.add_port(name="o2", port=bs_sup_izq2.ports["o2"])
combiner.add_port(name="o3", port=bs_sup_der2.ports["o2"])
combiner.add_port(name="o4", port=bs_inf_der2.ports["o2"])

combiner_ref = c.add_ref(combiner)
combiner_ref.connect("o3", brazo_inf.ports["o2"])

# — 4. PUERTOS EXTERNOS DEL MZI ——————————————————
c.add_port(name="o1", port=splitter_ref.ports["o1"])
c.add_port(name="o2", port=splitter_ref.ports["o2"])
c.add_port(name="o3", port=combiner_ref.ports["o2"])
c.add_port(name="o4", port=combiner_ref.ports["o1"])

c.draw_ports()

c.plot()
c.show()
```



Este es el interferómetro de Mach-Zehnder (MZI) 2x2 diseñado en Lab3, construido a partir de dos MMI 2x2 con curvas en S como divisor y combinador. Utiliza las mismas dimensiones de MMI del Lab2: W_MMI = 6.6 µm, L_MMI = 34.718 µm, W_wg = 1.8 µm y L_wg = 10.0 µm, con curvas en S de dx_bend = 30.0 µm y dy_bend = 40.0 µm que separan los puertos ±40.0 µm respecto al eje central. Los dos brazos del interferómetro tienen longitud base L_brazo = 400.0 µm.

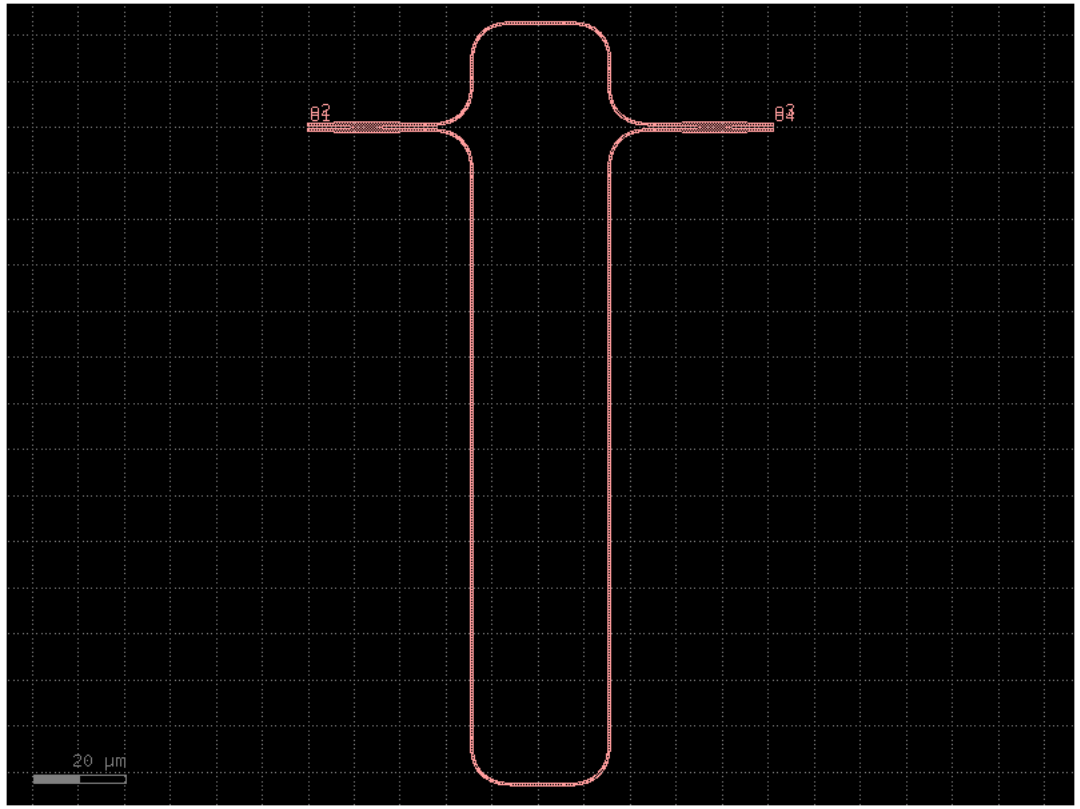
Financiado por la Unión Europea NextGenerationEU
GOBIERNO DE ESPAÑA MINISTERIO PARA LA TRANSFORMACIÓN DIGITAL Y DE LA FUNCIÓN PÚBLICA
Plan de Recuperación, Transformación y Resiliencia
España | digital 2026
PERTE Chip
UNIVERSITAT POLITÈCNICA DE VALÈNCIA
TSI-069100-2023-0002

Learning outcome #1b – Notebook section 1. Technology – MZI unbalanced

"Use your design results from Lab2 and Lab3 to create layout instances of your designed components: DCs, MMIs, MZIs & Ring Resonators"

```
# FSR = 5 nm  → dL ≈ 240 µm
c = gf.components.mzi2x2_2x2( delta_length=240,
        cross_section='strip')
c.draw_ports()
c.plot()
```

Este es el interferómetro de Mach-Zehnder (MZI) 2x2 generado directamente con el componente estándar de gdsfactory mzi2x2_2x2, con un desequilibrio de brazos delta_length = 240 µm diseñado para obtener un Free Spectral Range (FSR) de aproximadamente 5 nm.

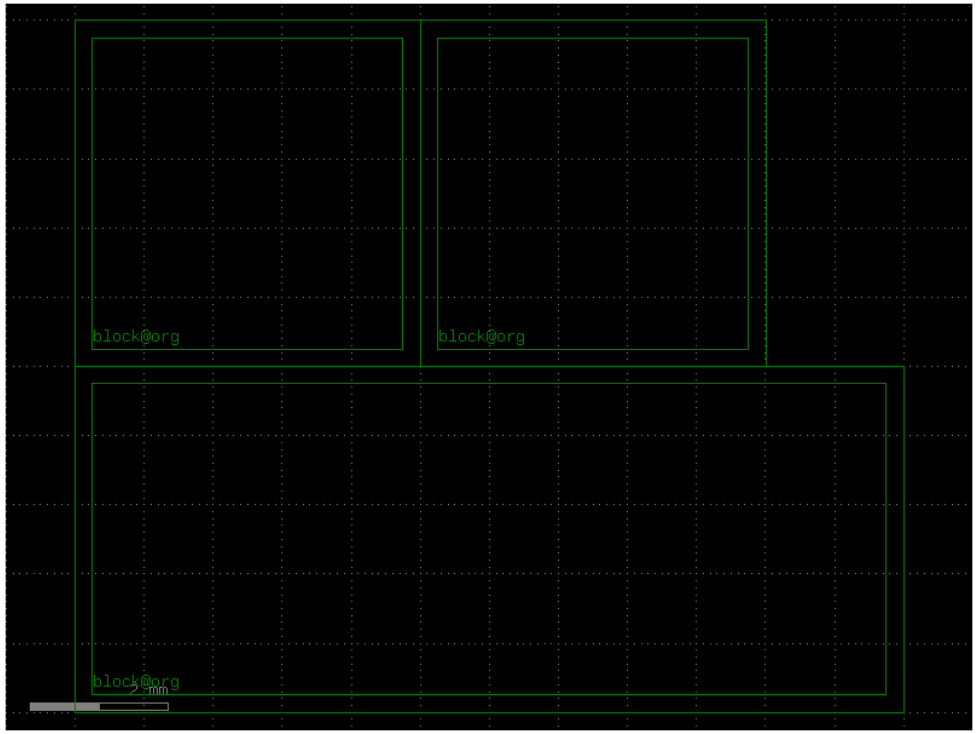


Financiado por la Unión Europea NextGenerationEU
GOBIERNO DE ESPAÑA MINISTERIO PARA LA TRANSFORMACIÓN DIGITAL Y DE LA FUNCIÓN PÚBLICA
Plan de Recuperación, Transformación y Resiliencia
España | digital 2026
PERTE Chip
UNIVERSITAT POLITÈCNICA DE VALÈNCIA
TSI-069100-2023-0002

Learning outcome #2a – Notebook section 2. Layout Fundamentals

“Once that we have a working 'new' component, we shall convert it into a Cell. This will allow us to have a hierarchical design”

```python
# 1. We define the cell as a function with it's corresponding parameters (and defaults)
gf.clear_cache()

@gf.cell
def die(dieL = 5000, dieW = 5000, border = 250, layer_box = "FLOORPLAN"):
    # Die specifications (Chip)
    box = gf.Component()
    obox = box.add_ref(gf.components.rectangle(size=(dieW,dieL),layer=layer_box))
    ibox = box.add_ref(gf.components.rectangle(size=(dieW-border*2,dieL-border*2),
                                               layer=layer_box)).dmovex(border).dmovey(border)
    box = gf.boolean(A=obox, B=ibox, operation="A-B", layer=layer_box)
    # Adding ports to a component
    box.add_port(name="block@org", center=[border,border], width=1, orientation=0, layer=layer_box)
    box.draw_ports()
    return box

# 2. We instantiate references of each cell. We can also name it individually to avoid problems
wafer = gf.Component()
dieW1 = 5000
dieW2 = 10000

c1 = wafer.add_ref(die(dieW = dieW))
c2 = wafer.add_ref(die(dieW = dieW))
c3 = wafer.add_ref(die(dieW = 12000))

c2.dmovex(dieW1)
c3.dmovey(-5000)
wafer.show()
wafer.plot()
```



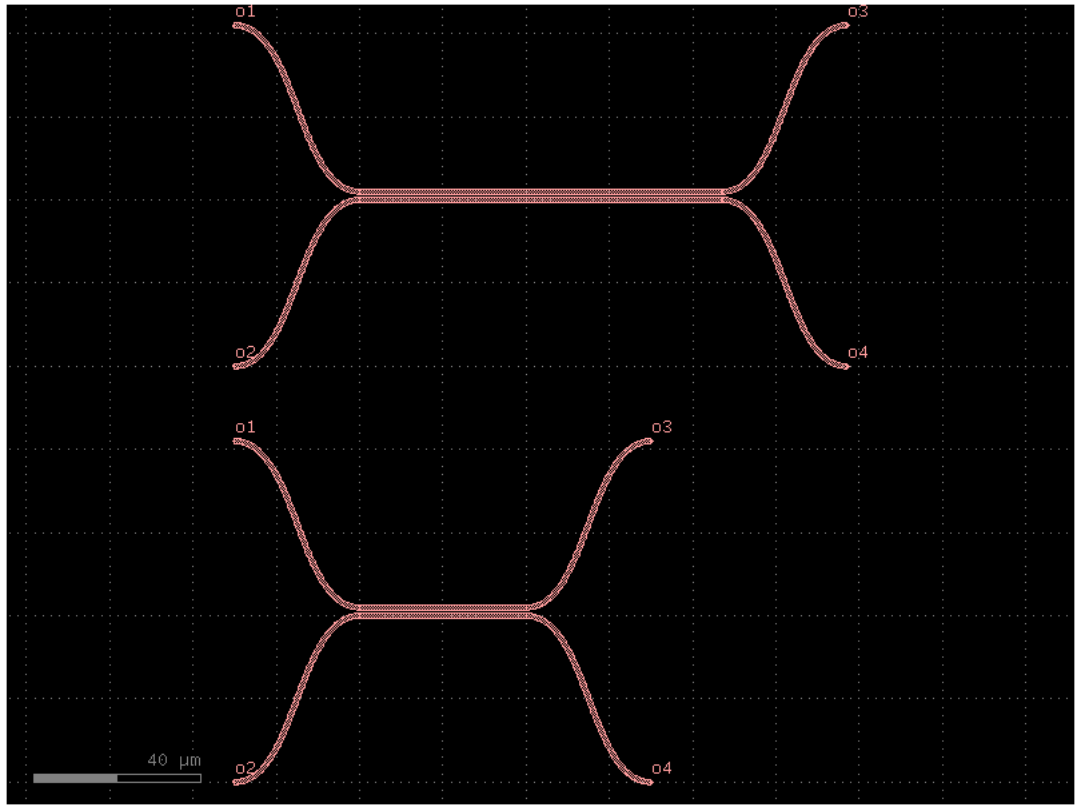
Este es el die (chip) base del diseño, definido como una celda. Consiste en un rectangulo hueco con unbored exterior de dimensiones dieW × dieL y uno interior desplazado 250 µm por cada lado, dejando un marco de 250 µm en la capa FLOORPLAN. En este bloque se instancian tres dies: dos de 5000 × 5000 µm colocados uno al lado del otro en X, y un tercero de 12000 × 5000 µm desplazado 5000 µm hacia abajo en Y.

Financiado por la Unión Europea NextGenerationEU
Plan de Recuperación, Transformación y Resiliencia
España | digital 2026
PERTE Chip
UNIVERSITAT POLITÈCNICA DE VALÈNCIA
TSI-069100-2023-0002

Learning outcome #2a – Notebook section 2. Layout Fundamentals – DC component cell

En este bloque se definen los componentes como celdas paramétricas reutilizables mediante @gf.cell. Nos permite instanciar cada componente múltiples veces con distintos parámetros.

```python
# 1. We define the cell as a function with it's corresponding parameters (and defaults)
gf.clear_cache()

@gf.cell
def dc_2x2(
    length: float = 86.982,
    gap: float = 0.8,
    dx: float = 30.0,
    dy: float = 40.0,
    width: float = 1.2,):

    # Creamos el componente
    c = gf.Component()
    # Definimos la sección transversal tipo Strip con  ancho de 1.2 micras en la capa 'WG'
    xs = gf.cross_section.strip(width=1.2, layer="WG")

    region_central = c.add_ref(gf.components.coupler_straight( length=length, gap=gap,cross_section=xs))
    #izquierda
    s_bend_sup_izq = c.add_ref(gf.components.bend_s(size=(dx, dy), cross_section=xs))
    s_bend_sup_izq.connect("o1", region_central.ports["o1"])
    s_bend_inf_izq = c.add_ref(gf.components.bend_s( size=(dx, dy), cross_section=xs)).mirror()
    s_bend_inf_izq.connect("o1", region_central.ports["o2"])
    #derecha
    s_bend_sup_der = c.add_ref( gf.components.bend_s(size=(dx, dy), cross_section=xs))
    s_bend_sup_der.connect("o1", region_central.ports["o3"])
    s_bend_inf_der = c.add_ref(gf.components.bend_s(size=(dx, dy), cross_section=xs)).mirror()
    s_bend_inf_der.connect("o1", region_central.ports["o4"])
    #puertos
    c.add_port(name="o1", port=s_bend_inf_izq.ports["o2"]) # Entrada Inferior Izquierda
    c.add_port(name="o2", port=s_bend_sup_izq.ports["o2"]) # Entrada Superior Izquierda
    c.add_port(name="o3", port=s_bend_sup_der.ports["o2"]) # Salida Superior Derecha
    c.add_port(name="o4", port=s_bend_inf_der.ports["o2"]) # Salida Inferior Derecha

    c.draw_ports()
    return c

# 2. We instantiate references of each cell. We can also name it individually to avoid problems
chip_dc = gf.Component()

dc1 = chip_dc.add_ref(dc_2x2( length = 40 ))
dc2= chip_dc.add_ref(dc_2x2())

dc2.dmovey(100)
chip_dc.plot()
```

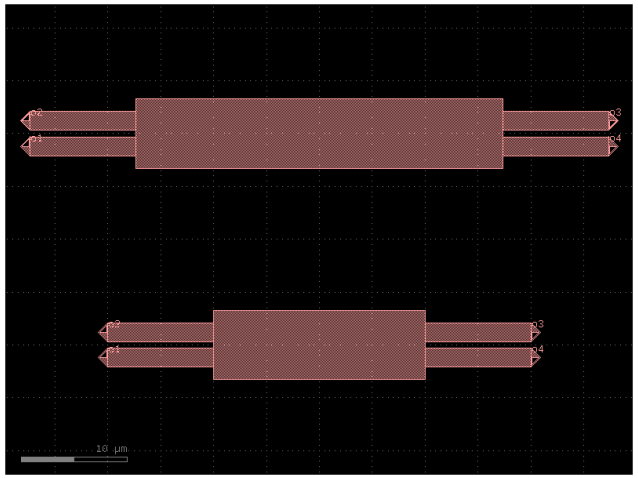


Financiado por la Unión Europea NextGenerationEU
TSI-069100-2023-0002

Learning outcome #2a – Notebook section 2. Layout Fundamentals – MMI cell

En este bloque se definen los componentes como celdas paramétricas reutilizables mediante @gf.cell. Nos permite instanciar cada componente múltiples veces con distintos parámetros.

```python
gf.clear_cache()

@gf.cell
def mmi2x2(
    W_MMI   = 6.6,
    L_MMI   = 34.718,
    W_wg    = 1.8,
    L_wg    = 10.0,
    dy      = 0.1,
):
    c = gf.Component()

    # Posiciones de los puertos:
    pos_inf = W_MMI * (-1/6) + (-dy)   # Puerto inferior
    pos_sup = W_MMI * ( 1/6) + ( dy)   # Puerto superior

    # Bloque central
    mmi_core = c.add_ref(
        gf.components.rectangle(size=(L_MMI, W_MMI), layer="WG"))
    # Centrar el bloque en el origen
    mmi_core.dmove((-L_MMI / 2, -W_MMI / 2))

    # Guias de acceso
    xs = gf.cross_section.strip(width = W_wg, layer = 'WG')
    wg = gf.components.straight(length = L_wg, cross_section=xs)

    # Entrada superior izquierda (o2)
    t_o2 = c.add_ref(wg)
    t_o2.drotate(180)
    t_o2.dmove((-L_MMI / 2, pos_sup))

    # Entrada inferior izquierda (o1)
    t_o1 = c.add_ref(wg)
    t_o1.drotate(180)
    t_o1.dmove((-L_MMI / 2, pos_inf))

    # Salida superior derecha (o3)
    t_o3 = c.add_ref(wg)
    t_o3.dmove((L_MMI / 2, pos_sup))

    # Salida inferior derecha (o4)
    t_o4 = c.add_ref(wg)
    t_o4.dmove((L_MMI / 2, pos_inf))
```



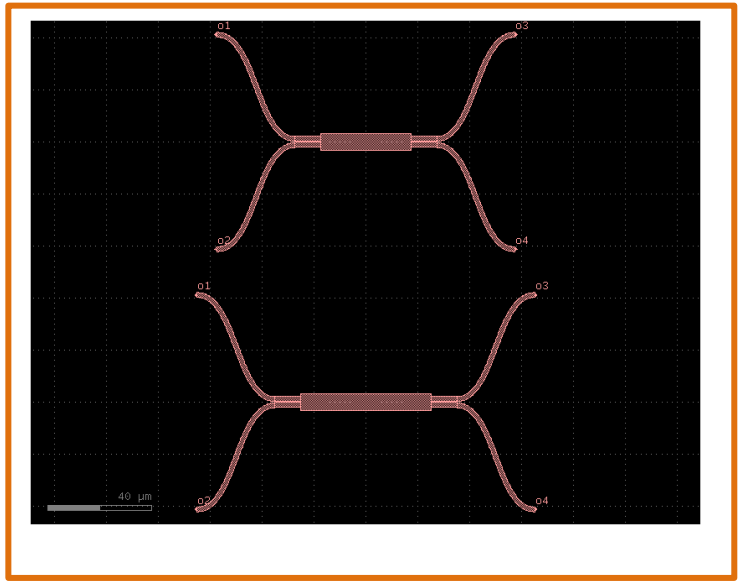
```python
    #  Puertos externos
    c.add_port(name="o1", center=(-L_MMI/2 - L_wg, pos_inf),width=W_wg, orientation=180, layer="WG")
    c.add_port( name="o2", center=(-L_MMI/2 - L_wg, pos_sup),width=W_wg, orientation=180, layer="WG")
    c.add_port(name="o3", center=( L_MMI/2 + L_wg, pos_sup),width=W_wg, orientation=0,   layer="WG")
    c.add_port(name="o4", center=( L_MMI/2 + L_wg, pos_inf),width=W_wg, orientation=0,   layer="WG")

    c.draw_ports()
    return c

# 2. We instantiate references of each cell. We can also name it individually to avoid problems
chip_mmi2x2 = gf.Component()

mmi1 = chip_mmi2x2.add_ref(mmi2x2(L_MMI  = 20  ))
mmi2 = chip_mmi2x2.add_ref(mmi2x2())
mmi2.dmovey(20)
```


Financiado por la Unión Europea NextGenerationEU
Plan de Recuperación, Transformación y Resiliencia
España | digital 2026
PERTE Chip
UNIVERSITAT POLITÈCNICA DE VALÈNCIA
TSI-069100-2023-0002

Learning outcome #2a – Notebook section 2. Layout Fundamentals – MMI bend cell

En este bloque se definen los componentes como celdas paramétricas reutilizables mediante @gf.cell. Nos permite instanciar cada componente múltiples veces con distintos parámetros.

```python
gf.clear_cache()
@gf.cell
def mmi2x2_bend(
    W_MMI   = 6.6,      # Ancho del bloque multimodo (mmi_Width del Lab2)
    L_MMI   = 34.718,   # L_MMI + dL_MMI = 34.518 + 0.2 del Lab2
    W_wg    = 1.8,      # Ancho guías de acceso (IO wg width del Lab2) +0.8
    L_wg   = 10.0,
    dy      = 0.1,      # Offset de posición de los puertos (dy del Lab2)
    dx_bend = 30.0,
    dy_bend = 40.0,
):
    c = gf.Component()
    # Posiciones de los puertos:
    pos_inf = W_MMI * (-1/6) + (-dy)   # Puerto inferior
    pos_sup = W_MMI * ( 1/6) + ( dy)   # Puerto superior

    # Bloque central
    mmi_core = c.add_ref(
        gf.components.rectangle(size=(L_MMI, W_MMI), layer="WG"))
    # Centrar el bloque en el origen
    mmi_core.dmove((-L_MMI / 2, -W_MMI / 2))

    # Guias de acceso
    xs = gf.cross_section.strip(width = W_wg, layer = 'WG')
    wg = gf.components.straight(length = L_wg, cross_section=xs)

    # Entrada superior izquierda (o2)
    t_o2 = c.add_ref(wg)
    t_o2.drotate(180)
    t_o2.dmove((-L_MMI / 2, pos_sup))

    # Entrada inferior izquierda (o1)
    t_o1 = c.add_ref(wg)
    t_o1.drotate(180)
    t_o1.dmove((-L_MMI / 2, pos_inf))

    # Salida superior derecha (o3)
    t_o3 = c.add_ref(wg)
    t_o3.dmove((L_MMI / 2, pos_sup))

    # Salida inferior derecha (o4)
    t_o4 = c.add_ref(wg)
    t_o4.dmove((L_MMI / 2, pos_inf))

    #  BEND_S conectados al final de cada guía
    # Izquierda superior
    bs_sup_izq = c.add_ref(gf.components.bend_s(size=(dx_bend, dy_bend), cross_section=xs))
    bs_sup_izq.connect("o1", t_o1.ports["o2"])

    # Izquierda inferior
    bs_inf_izq = c.add_ref(gf.components.bend_s(size=(dx_bend, dy_bend), cross_section=xs)).mirror()
    bs_inf_izq.connect("o1", t_o2.ports["o2"])

    # Derecha superior
    bs_sup_der = c.add_ref(gf.components.bend_s(size=(dx_bend, dy_bend), cross_section=xs))
    bs_sup_der.connect("o1", t_o3.ports["o2"])

    # Derecha inferior
    bs_inf_der = c.add_ref(gf.components.bend_s(size=(dx_bend, dy_bend), cross_section=xs)).mirror()
    bs_inf_der.connect("o1", t_o4.ports["o2"])

    # Puertos al final de los bend_s
    c.add_port(name="o1", port=bs_inf_izq.ports["o2"])  # Entrada inferior izq
    c.add_port(name="o2", port=bs_sup_izq.ports["o2"])  # Entrada superior izq
    c.add_port(name="o3", port=bs_sup_der.ports["o2"])  # Salida superior der
    c.add_port(name="o4", port=bs_inf_der.ports["o2"])  # Salida inferior der
    c.draw_ports()
    return c

# 2. We instantiate references of each cell. We can also name it individually to avoid problems
chip_mmi2x2_bend = gf.Component()

mmi1_b = chip_mmi2x2_bend.add_ref(mmi2x2_bend(L_MMI   = 50))
mmi2_b = chip_mmi2x2_bend.add_ref(mmi2x2_bend())
mmi2_b.dmovey(100)

chip_mmi2x2_bend.plot()
```

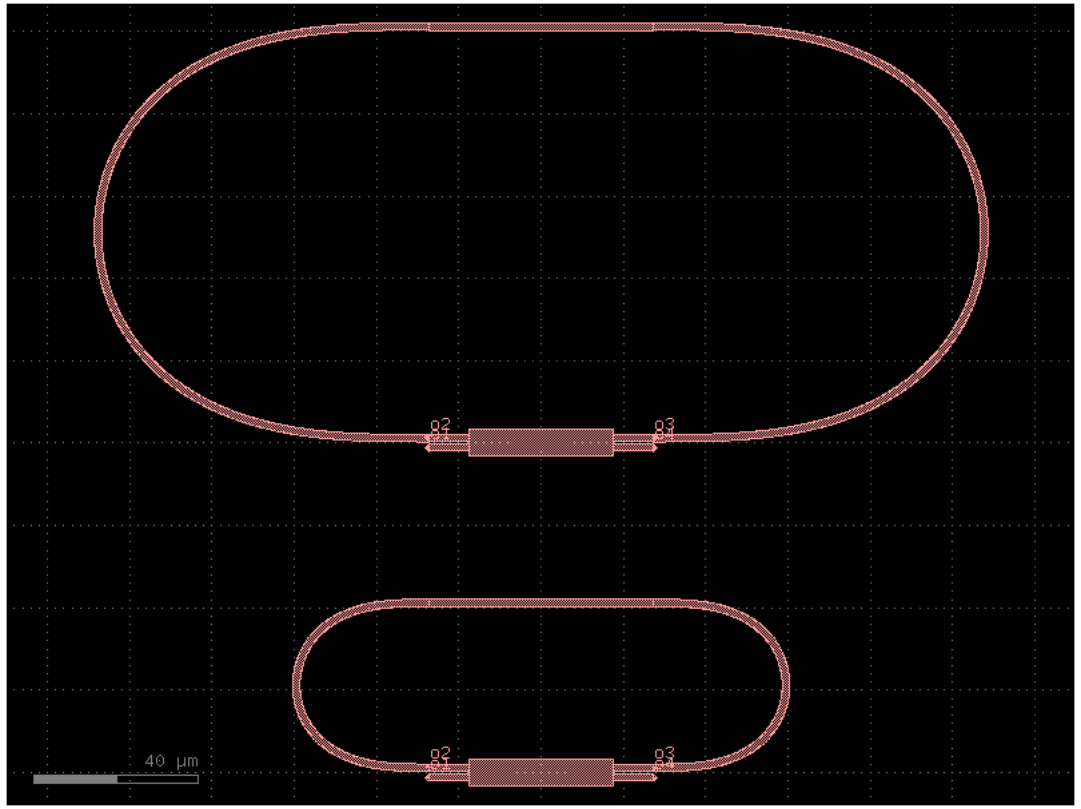


Financiado por la Unión Europea NextGenerationEU
Plan de Recuperación, Transformación y Resiliencia
España | digital 2026
PERTE Chip
UNIVERSITAT POLITÈCNICA DE VALÈNCIA
TSI-069100-2023-0002

Learning outcome #2a – Notebook section 2. Layout Fundamentals – Ring cell

En este bloque se definen los componentes como celdas paramétricas reutilizables mediante @gf.cell. Nos permite instanciar cada componente múltiples veces con distintos parámetros.

```python
gf.clear_cache()

@gf.cell
def ring_resonator_r(
    W_MMI   = 6.6,
    L_MMI   = 34.718,
    W_wg    = 1.8,
    L_wg    = 10.0,
    dy      = 0.1,
    radio   = 50.0,
    L_extra = 0.0,
):
    c  = gf.Component()
    xs = gf.cross_section.strip(width=W_wg, layer="WG")

    # MMI central
    mmi = c.add_ref(mmi2x2(W_MMI=W_MMI, L_MMI=L_MMI, W_wg=W_wg, L_wg=L_wg, dy=dy))

    # Lado derecho
    bend_der_1 = c.add_ref(gf.components.bend_euler(angle=180, radius=radio, cross_section=xs))
    bend_der_1.connect("o1", mmi.ports["o3"])

    # Lado izquierdo
    bend_izq_1 = c.add_ref(gf.components.bend_euler(angle=-180, radius=radio, cross_section=xs))
    bend_izq_1.connect("o1", mmi.ports["o2"])

    # Separación entre o3 y o2
    sep_puertos = abs(mmi.ports["o2"].dy - mmi.ports["o3"].dy)

    # Recto superior
    recto = c.add_ref(gf.components.straight(length=sep_puertos , cross_section=xs))
    recto.connect("o1", bend_der_1.ports["o2"])

    gf.routing.route_single(component=c,port1=recto.ports["o2"],port2=bend_izq_1.ports["o2"],cross_section=xs)

    # Puertos externos
    c.add_port(name="o1", port=mmi.ports["o1"])
    c.add_port(name="o2", port=mmi.ports["o2"])
    c.add_port(name="o3", port=mmi.ports["o3"])
    c.add_port(name="o4", port=mmi.ports["o4"])

    c.draw_ports()
    return c

# 2. We instantiate references of each cell. We can also name it individually to avoid problems
chip_ring_resonator = gf.Component()

ring1 = chip_ring_resonator.add_ref(ring_resonator_r( radio   = 20.0))
ring2 = chip_ring_resonator.add_ref(ring_resonator_r())
ring2.dmovey(80)

chip_ring_resonator.plot()
```

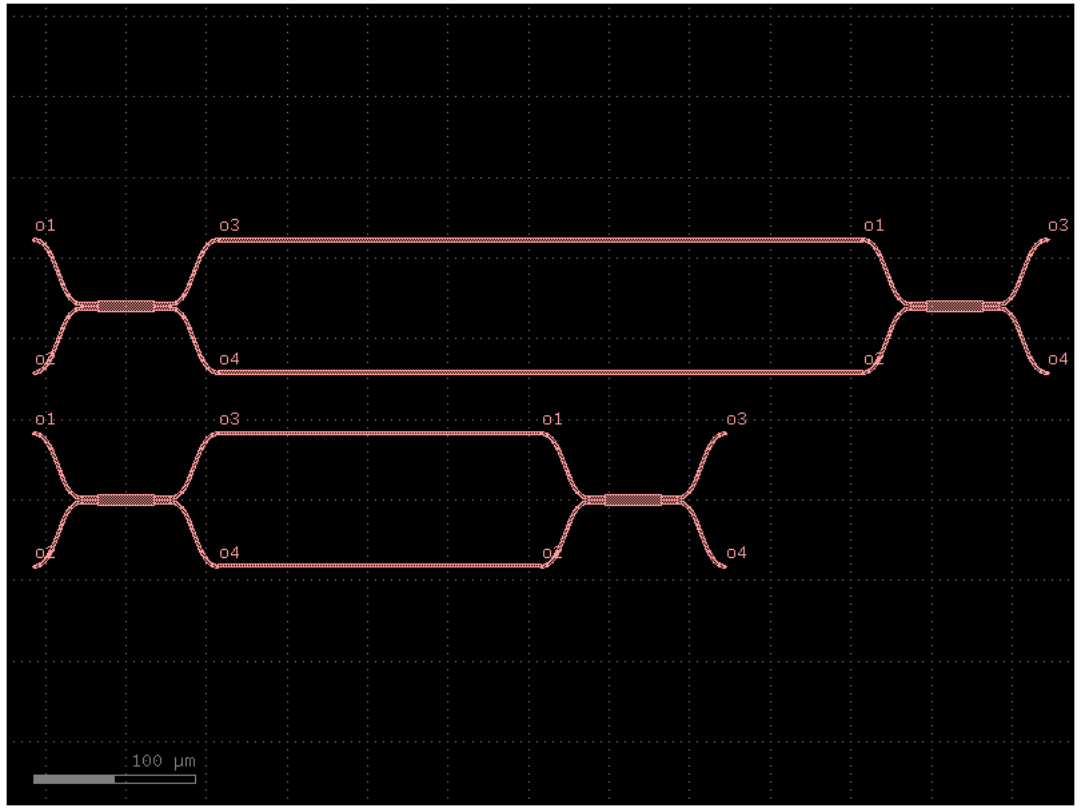


Financiado por la Unión Europea NextGenerationEU
TSI-069100-2023-0002

Learning outcome #2a – Notebook section 2. Layout Fundamentals – MZI cell

En este bloque se definen los componentes como celdas paramétricas reutilizables mediante @gf.cell. Nos permite instanciar cada componente múltiples veces con distintos parámetros.

```python
gf.clear_cache()

@gf.cell
def mzi(
    delta_length= 100.0,
    L_brazo=400.0,
):
    c  = gf.Component()
    xs = gf.cross_section.strip(width=1.8, layer="WG")

    # MMI izquierdo
    MMI_izq = c.add_ref(mmi2x2_bend())

    # Brazos del MZI

    brazo_sup = c.add_ref(gf.components.straight(length=L_brazo, cross_section=xs)) #length=L_brazo + delta_length
    brazo_inf = c.add_ref(gf.components.straight(length=L_brazo, cross_section=xs))

    # Conectar brazos a las salidas del MMI
    brazo_sup.connect("o1", MMI_izq.ports["o3"])  # Sale por o3 (superior der)
    brazo_inf.connect("o1", MMI_izq.ports["o4"])  # Sale por o4 (inferior der)

    # MMI derecho
    MMI_der = c.add_ref(mmi2x2_bend())
    MMI_der.connect("o2", brazo_inf.ports["o2"])  # o2 recibe brazo inferior


    # Puertos MZI
    c.add_port(name="o1", port=MMI_izq.ports["o1"])   # Entrada inferior izq
    c.add_port(name="o2", port=MMI_izq.ports["o2"])   # Entrada superior izq
    c.add_port(name="o3", port=MMI_der.ports["o3"])   # Salida superior der
    c.add_port(name="o4", port=MMI_der.ports["o4"])   # Salida inferior der

    c.draw_ports()
    return c

# 2. We instantiate references of each cell. We can also name it individually to avoid problems
chip_MZI = gf.Component()

mzi1 = chip_MZI.add_ref(mzi( L_brazo=200.0,))
mzi2 = chip_MZI.add_ref(mzi())
mzi2.dmovey(120)

chip_MZI.plot()
```

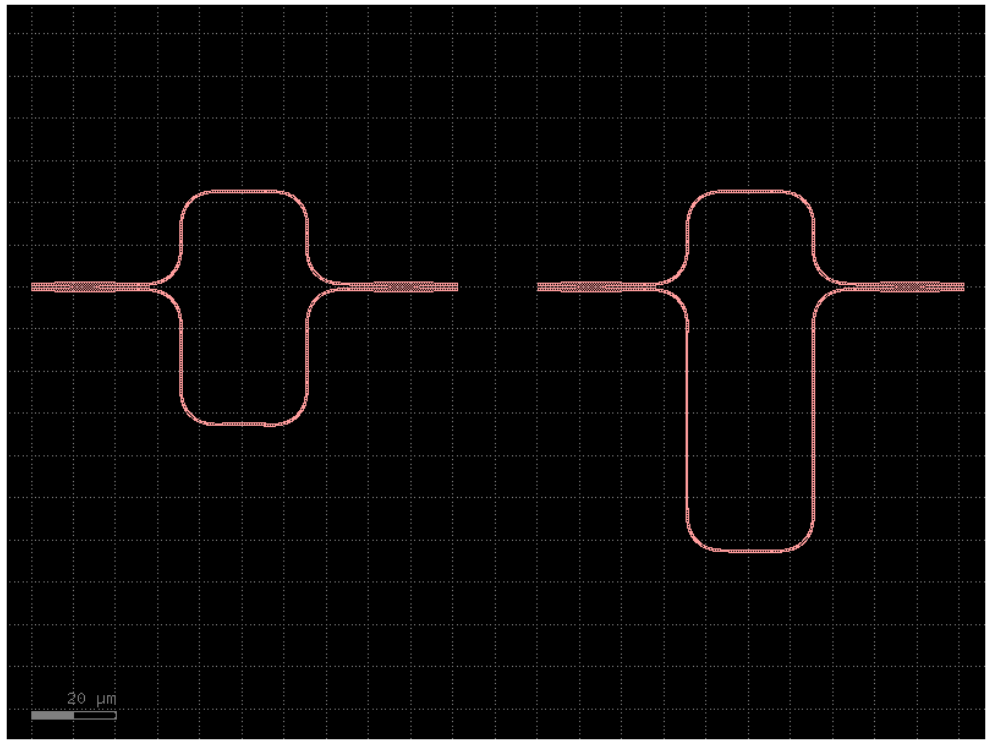


Financiado por la Unión Europea NextGenerationEU
Plan de Recuperación, Transformación y Resiliencia
España | digital 2026
PERTE Chip
UNIVERSITAT POLITÈCNICA DE VALÈNCIA
TSI-069100-2023-0002

Learning outcome #2a – Notebook section 2. Layout Fundamentals – MZI unbalanced cell

En este bloque se definen los componentes como celdas paramétricas reutilizables mediante @gf.cell. Nos permite instanciar cada componente múltiples veces con distintos parámetros.

```python
import gdsfactory as gf

@gf.cell
def mzi_unbalanced_2x2(delta_length=0.0):
    c = gf.Component()

    mzi_ref = c.add_ref(gf.components.mzi2x2_2x2(
        delta_length=delta_length,
        cross_section='strip'
    ))
    c.add_ports(mzi_ref.ports)

    return c

# Lo probamos
chip_MZI_u = gf.Component()

mzi1_u = chip_MZI_u.add_ref(mzi_unbalanced_2x2(delta_length=20.0))
mzi2_u = chip_MZI_u.add_ref(mzi_unbalanced_2x2(delta_length=80.0))
mzi2_u.dmovex(120)
chip_MZI_u.plot()
```



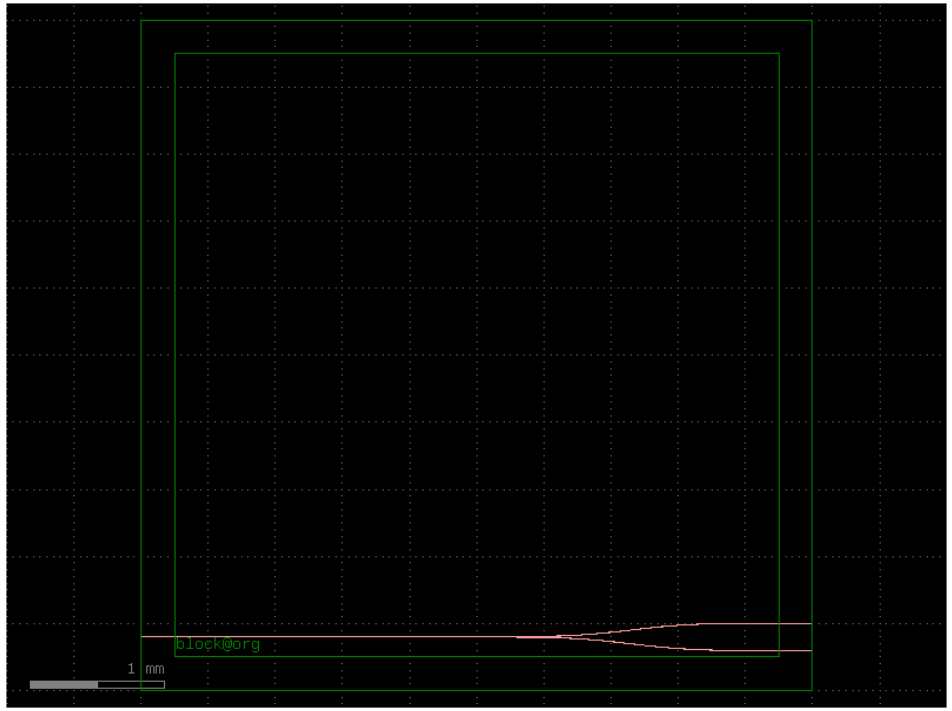
Financiado por la Unión Europea NextGenerationEU
Plan de Recuperación, Transformación y Resiliencia
España | digital 2026
PERTE Chip
UNIVERSITAT POLITÈCNICA DE VALÈNCIA
TSI-069100-2023-0002

Learning outcome #3 – Notebook section 3. Complete layout

Read and describe 4 code blocks in section 3

Python code block 1:

```
layer_wg  = "WG"
minrad = 50
dieW = 5000

main = gf.Component()

die_ref = main.add_ref(die(dieW = dieW, layer_box="FLOORPLAN"))

## Add first a test MMI routed from side to side
c_mmi = cells.mmi1x2()
mmi = main.add_ref(c_mmi)
mmi.dmovex(die_ref["block@org"].dx + 0.5*dieW).dmovey(die_ref['block@org'].dy + 150)

## Route waveguides from MMI to the die edges
xs = 'strip'
### First add the i/o waveguides - to be sure you 'cut' on a straight section
strin = (main.add_ref(gf.components.straight(length=500, cross_section='strip')).dmovex(0).dmovey(mmi['o1'].dy))
strout1 = (main.add_ref(gf.components.straight(length=500, cross_section=xs)).drotate(180).dmovey(mmi.ports['o3'].dy-100).dmovex(dieW))
strout2 = (main.add_ref(gf.components.straight(length=500, cross_section=xs)).drotate(180).dmovey(mmi.ports['o2'].dy+100).dmovex(dieW))
### Then route from the i/o waveguides to the MMI
gf.routing.route_single_sbend(main,port1=strin['o2'], port2=mmi['o1'], cross_section=xs)
gf.routing.route_single_sbend(main,port1=mmi['o2'], port2=strout2['o2'], cross_section=xs)
gf.routing.route_single_sbend(main,port1=mmi['o3'], port2=strout1['o2'], cross_section=xs)

main.plot()
main.show()
```



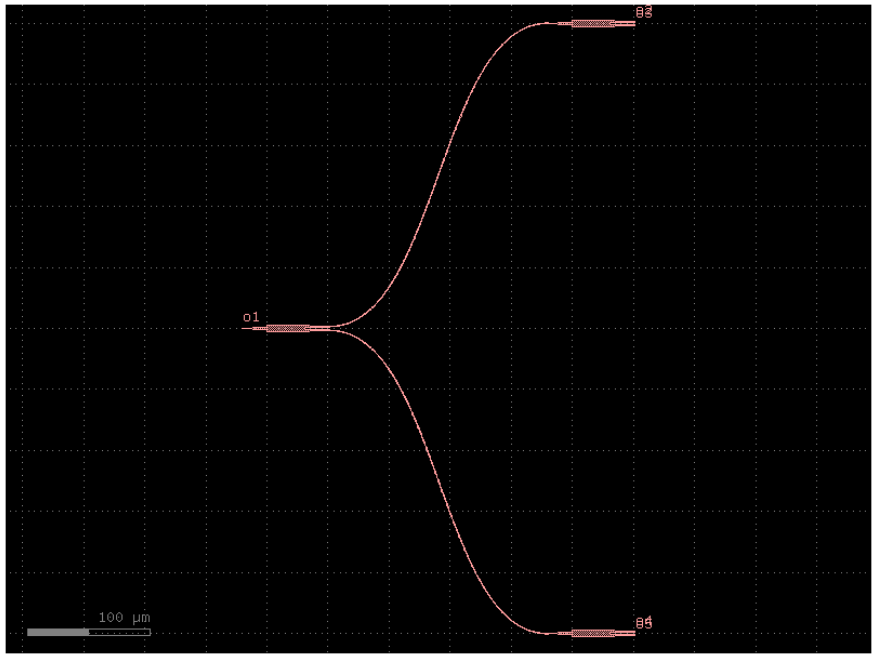
Financiado por la Unión Europea NextGenerationEU
GOBIERNO DE ESPAÑA MINISTERIO PARA LA TRANSFORMACIÓN DIGITAL Y DE LA FUNCIÓN PÚBLICA
Plan de Recuperación, Transformación y Resiliencia
España | digital 2026
PERTE Chip
UNIVERSITAT POLITÈCNICA DE VALÈNCIA
TSI-069100-2023-0002

Learning outcome #3 – Notebook section 3. Complete layout

Read and describe 4 code blocks in section 3

Python code block 2:

```python
# Let's create a MMI tree

c = gf.Component()
c_mmi = cells.mmi1x2()

mmi1 = c.add_ref(c_mmi)
mmi2 = c.add_ref(c_mmi)
mmi3 = c.add_ref(c_mmi)

mmi2.dmovex(250).dmovey(250)
mmi3.dmovex(250).dmovey(-250)

gf.routing.route_single_sbend(component=c, port1= mmi1['o2'], port2=mmi2['o1'],cross_section='strip')
gf.routing.route_single_sbend(component=c, port1= mmi1['o3'], port2=mmi3['o1'],cross_section='strip')
c.add_port(name='o1', port=mmi1['o1'])
c.add_port(name='o2', port=mmi2['o2'])
c.add_port(name='o3', port=mmi2['o3'])
c.add_port(name='o4', port=mmi3['o2'])
c.add_port(name='o5', port=mmi3['o3'])
c.draw_ports()
c.plot()
```



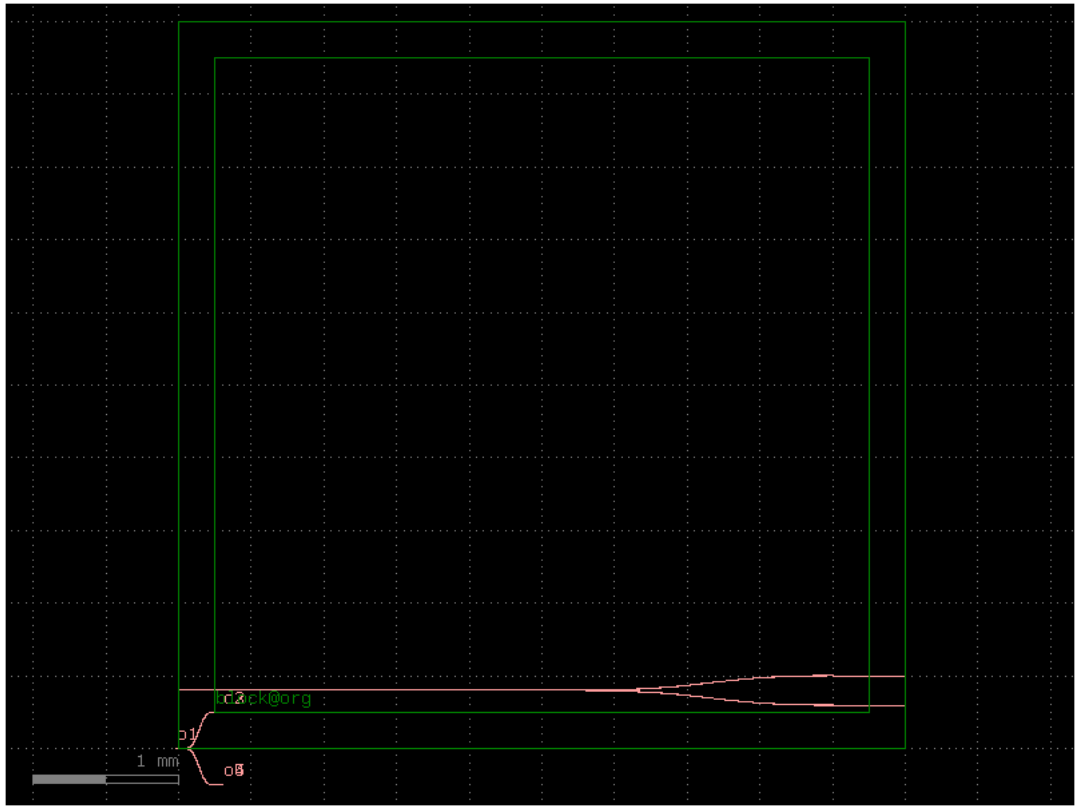
Financiado por la Unión Europea NextGenerationEU
TSI-069100-2023-0002

Learning outcome #3 – Notebook section 3. Complete layout

Read and describe 4 code blocks in section 3

Python code block 3:

```python
# Once we "test" the high-level component, we can create a cell for it
@gf.cell
def mmi_tree_1x4(pad_x = 250 ,pad_y = 250):
    c = gf.Component()
    c_mmi = cells.mmi1x2()
    mmi1 = c.add_ref(c_mmi)
    mmi2 = c.add_ref(c_mmi)
    mmi3 = c.add_ref(c_mmi)

    mmi2.dmovex(pad_x).dmovey(pad_y)
    mmi3.dmovex(pad_x).dmovey(-pad_y)

    gf.routing.route_single_sbend(component=c, port1= mmi1['o2'], port2=mmi2['o1'],cross_section='strip')
    gf.routing.route_single_sbend(component=c, port1= mmi1['o3'], port2=mmi3['o1'],cross_section='strip')
    c.add_port(name='o1', port=mmi1['o1'])
    c.add_port(name='o2', port=mmi2['o2'])
    c.add_port(name='o3', port=mmi2['o3'])
    c.add_port(name='o4', port=mmi3['o2'])
    c.add_port(name='o5', port=mmi3['o3'])
    c.draw_ports()
    return c


# With the cell created, we can instantiate and use it in our main component
cell_1x4 = main.add_ref(mmi_tree_1x4())
main.plot()
```



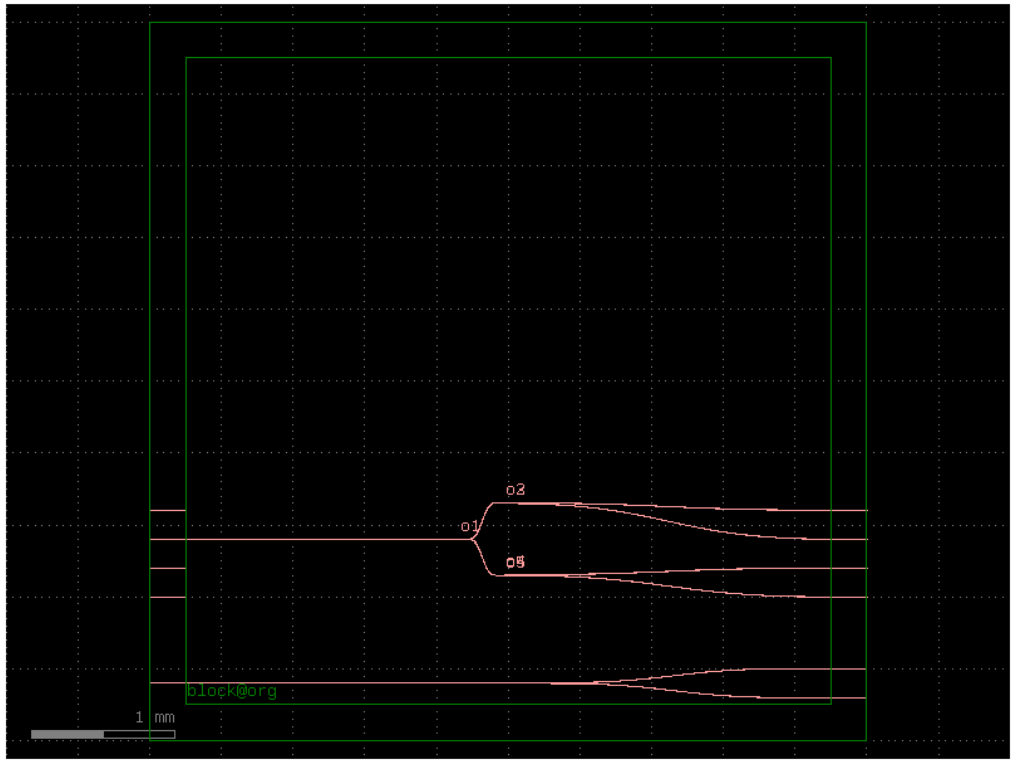
Financiado por la Unión Europea NextGenerationEU
TSI-069100-2023-0002

Learning outcome #3 – Notebook section 3. Complete layout

Read and describe 4 code blocks in section 3

Python code block 4:

```
## Arrayed waveguides and MMI tree

sp = 200
border = 250

in_arr = main.add_ref(
    gf.components.straight_array(n=4, spacing=sp, length=border, cross_section=xs)
).dmovey(
    1000
)  #!!!!!! Easy to put IO Waveguides for a complete design !!!!

out_arr = (
    main.add_ref(
        gf.components.straight_array(n=4, spacing=sp, length=border, cross_section=xs)
    )
    .dmovex(dieW - border)
    .dmovey(1000)
)

cell_1x4.dmovex(0.5*dieW - mmi_tree_1x4().dxsize).dmovey(in_arr['o6'].dy)
gf.routing.route_single_sbend(component=main, port1=in_arr['o6'], port2=cell_1x4['o1'])
gf.routing.route_single_sbend(component=main, port1=cell_1x4['o2'], port2=out_arr['o4'])
gf.routing.route_single_sbend(component=main, port1=cell_1x4['o3'], port2=out_arr['o3'])
gf.routing.route_single_sbend(component=main, port1=cell_1x4['o4'], port2=out_arr['o2'])
gf.routing.route_single_sbend(component=main, port1=cell_1x4['o5'], port2=out_arr['o1'])
main.plot()
main.show()
```

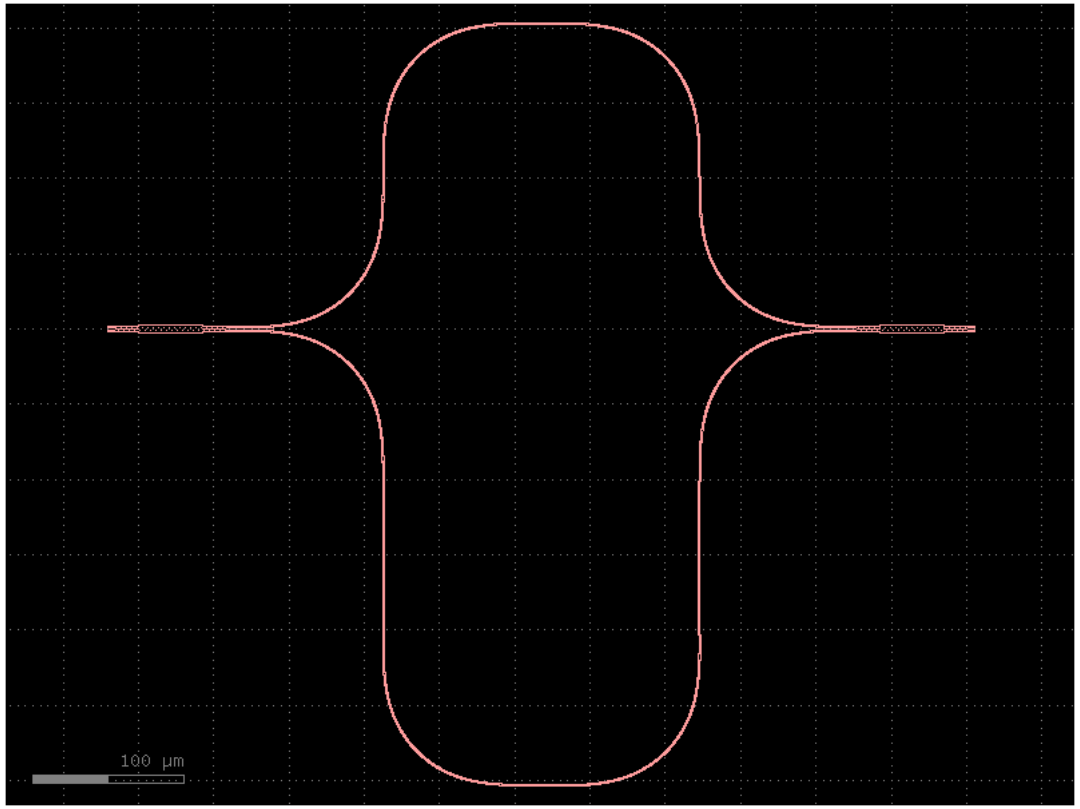


Financiado por la Unión Europea NextGenerationEU
TSI-069100-2023-0002

Learning outcome #4 – Notebook section 4. Exercises – Part a) Creating components

Code for …
- Create a cell component for a unbalanced MZI, using 2x2 50:50 MMIs with arm length difference as parameter

```python
import gdsfactory as gf

@gf.cell
def mzi_unbalanced_2x2(delta_length=0.0):
    c = gf.Component()

    mzi_ref = c.add_ref(gf.components.mzi2x2_2x2(
        delta_length=delta_length,
        cross_section='strip'
    ))
    c.add_ports(mzi_ref.ports)

    return c

# Lo probamos
mi_mzi_cell = mzi_unbalanced_2x2(delta_length=200)
mi_mzi_cell.plot()
```



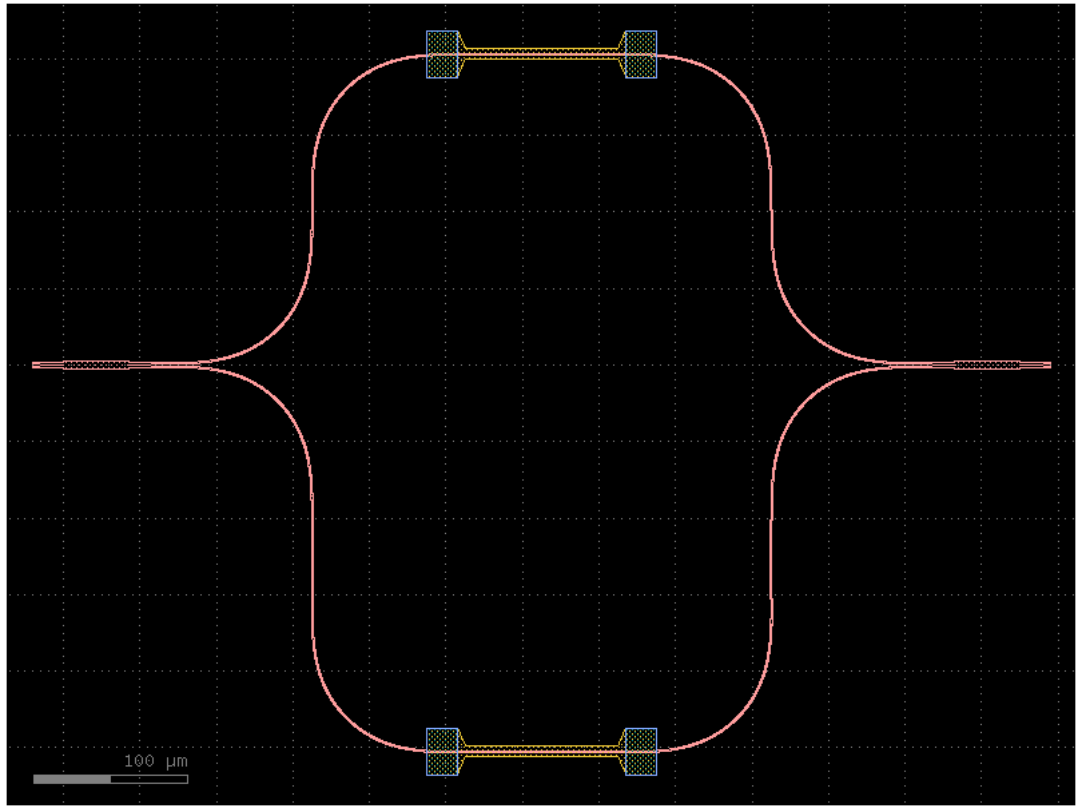
Financiado por la Unión Europea NextGenerationEU
TSI-069100-2023-0002

Learning outcome #4 – Notebook section 4. Exercises – Part a) Creating components

Code for …

- Create a cell component based on the previous, where the arms have thermal tuners on top of each one

```python
import gdsfactory as gf
from functools import partial

# Fijamos los parámetros del heater sin cross_section
straight_heater = partial(gf.components.straight_heater_metal, length=200)

@gf.cell
def mzi_heater_h(
    delta_length=100,
    length_x=100,
):
    c = gf.Component()
    mzi = c.add_ref(gf.components.mzi2x2_2x2(
        delta_length=delta_length,
        length_x=length_x,
        straight_x_top=straight_heater,
        straight_x_bot=straight_heater,
        cross_section='strip'
    ))
    c.add_ports(mzi.ports)
    return c

gf.clear_cache()
mzi_heatercell = mzi_heater_h(delta_length=100, length_x=100)
mzi_heatercell.plot()
```



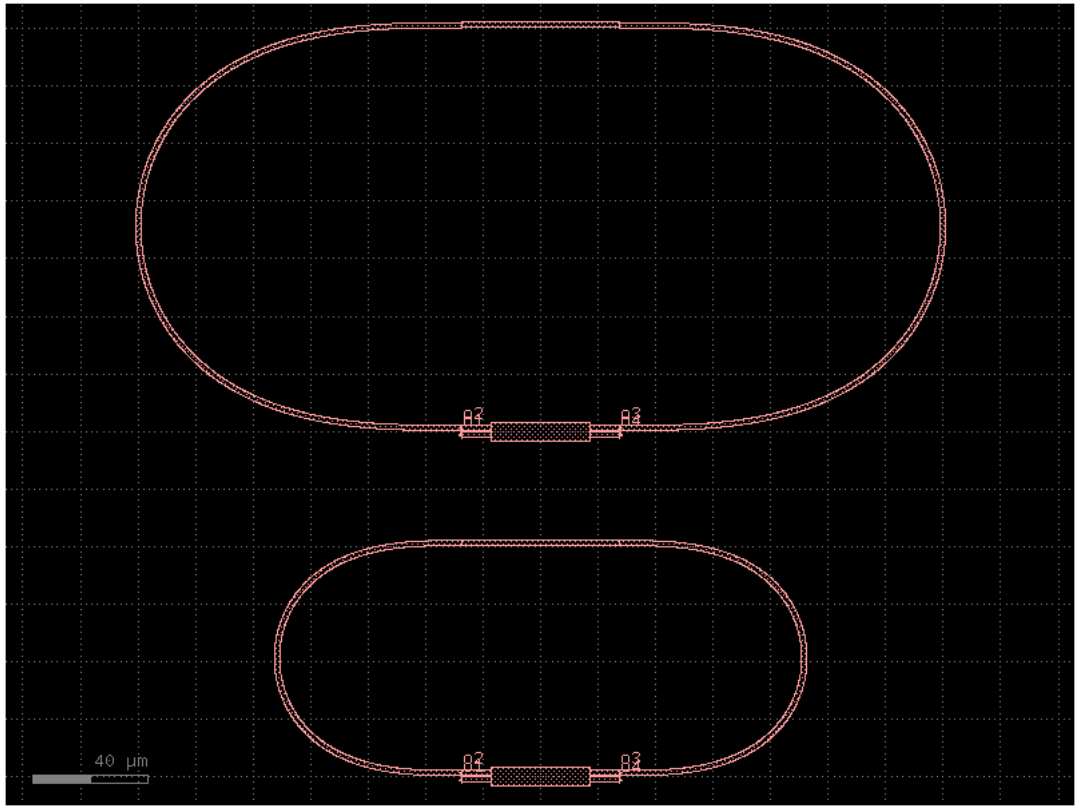
Financiado por la Unión Europea NextGenerationEU
Plan de Recuperación, Transformación y Resiliencia
España | digital 2026
PERTE Chip
UNIVERSITAT POLITÈCNICA DE VALÈNCIA
TSI-069100-2023-0002

Learning outcome #4 – Notebook section 4. Exercises – Part a) Creating components

Code for …

- Create a cell component for an all-pass ring resonator, using 2x2 50:50 MMIs, with extra length parameter for different perimeters

```python
gf.clear_cache()

@gf.cell
def ring_resonator_cell(
    W_MMI   = 6.6,
    L_MMI   = 34.718,
    W_wg    = 1.8,
    L_wg    = 10.0,
    dy      = 0.1,
    radio   = 20.0,
    L_extra = 0.0,
):
    c  = gf.Component()
    xs = gf.cross_section.strip(width=W_wg, layer="WG")

    # MMI central
    mmi = c.add_ref(mmi2x2(W_MMI=W_MMI, L_MMI=L_MMI, W_wg=W_wg, L_wg=L_wg, dy=dy))

    # Lado derecho
    bend_der_1 = c.add_ref(gf.components.bend_euler(angle=180, radius=radio + L_extra, cross_section=xs))
    bend_der_1.connect("o1", mmi.ports["o3"])

    # Lado izquierdo
    bend_izq_1 = c.add_ref(gf.components.bend_euler(angle=-180, radius=radio + L_extra, cross_section=xs))
    bend_izq_1.connect("o1", mmi.ports["o2"])

    # Separación entre o3 y o2
    sep_puertos = abs(mmi.ports["o2"].dy - mmi.ports["o3"].dy)

    # Recto superior
    recto = c.add_ref(gf.components.straight(length=sep_puertos , cross_section=xs))
    recto.connect("o1", bend_der_1.ports["o2"])
    # Si no coincide exactamente usar route_single:
    gf.routing.route_single(component=c,port1=recto.ports["o2"],port2=bend_izq_1.ports["o2"],cross_section=xs)

    # Puertos externos
    c.add_port(name="o1", port=mmi.ports["o1"])
    c.add_port(name="o2", port=mmi.ports["o2"])
    c.add_port(name="o3", port=mmi.ports["o3"])
    c.add_port(name="o4", port=mmi.ports["o4"])

    c.draw_ports()
    return c

# 2. We instantiate references of each cell. We can also name it individually to avoid problems
chip_ring_resonator = gf.Component()

ring1 = chip_ring_resonator.add_ref(ring_resonator_cell(L_extra  = 20.0))
ring2 = chip_ring_resonator.add_ref(ring_resonator_cell(L_extra  = 50.0))
ring2.dmovey(120)

chip_ring_resonator.plot()
```



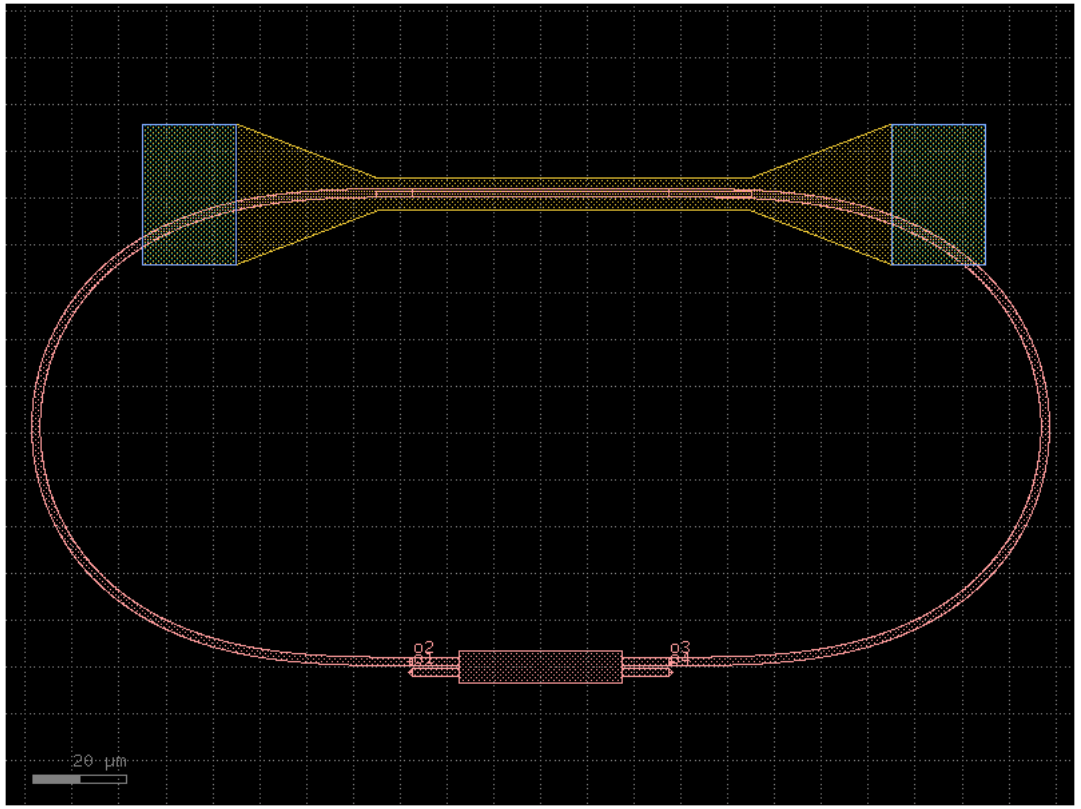
Financiado por la Unión Europea NextGenerationEU
TSI-069100-2023-0002

Learning outcome #4 – Notebook section 4. Exercises – Part a) Creating components

Code for …

- Create a cell component based on the previous, where the ring has a thermal tuner on top along all the perimeter

```python
@gf.cell
def ring_heater_cell(radio=50.0):
    c = gf.Component()

    ring = c.add_ref(ring_resonator_cell(radio=radio))

    recto_h = c.add_ref(cells.straight_heater_metal(length=80))
    recto_h.dmovex(ring.dx - 70 / 2)
    recto_h.dmovey(ring.dymax - 1.2)

    # Puertos externos
    c.add_port(name="o1", port=ring.ports["o1"])
    c.add_port(name="o2", port=ring.ports["o2"])
    c.draw_ports()
    return c

heatercell_p = ring_heater_cell()
heatercell_p.plot()
```

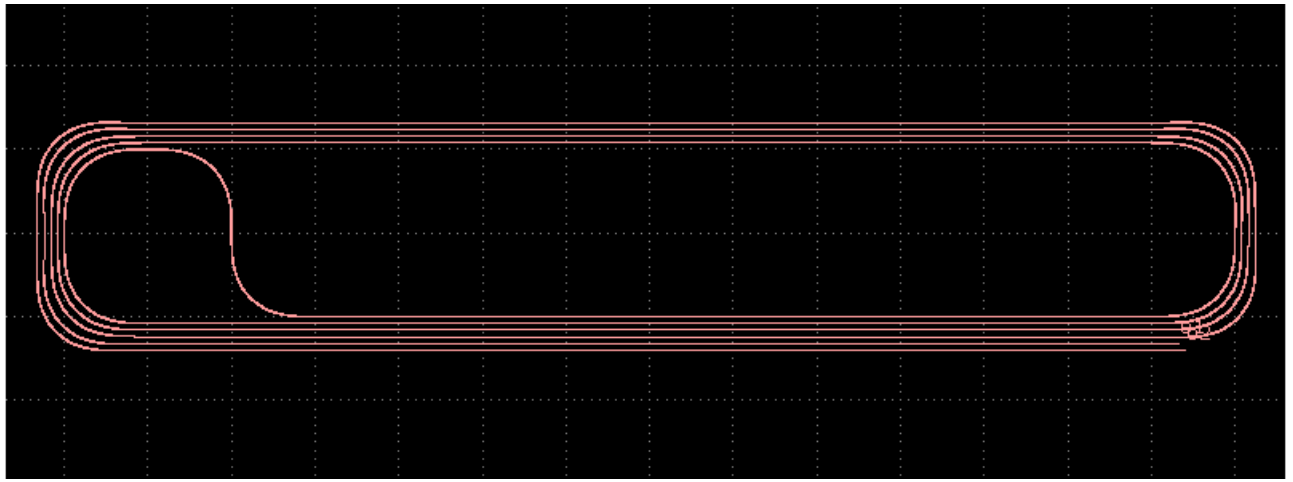


Financiado por la Unión Europea NextGenerationEU
GOBIERNO DE ESPAÑA MINISTERIO PARA LA TRANSFORMACIÓN DIGITAL Y DE LA FUNCIÓN PÚBLICA
Plan de Recuperación, Transformación y Resiliencia
España | digital 2026
PERTE Chip
UNIVERSITAT POLITÈCNICA DE VALÈNCIA
TSI-069100-2023-0002

Learning outcome #4 – Notebook section 4. Exercises – Part a) Creating components

Code for …
- Create a cell component based on existing GDSfactory spiral components, with length as parameter

```
def spiral(length=50,spacing=8, n_loops=5):
    c = gf.components.spiral(length=length, spacing=spacing,n_loops=n_loops, cross_section="strip")
    c.draw_ports()
    return c

c = spiral(length=1000)
c.plot()
```

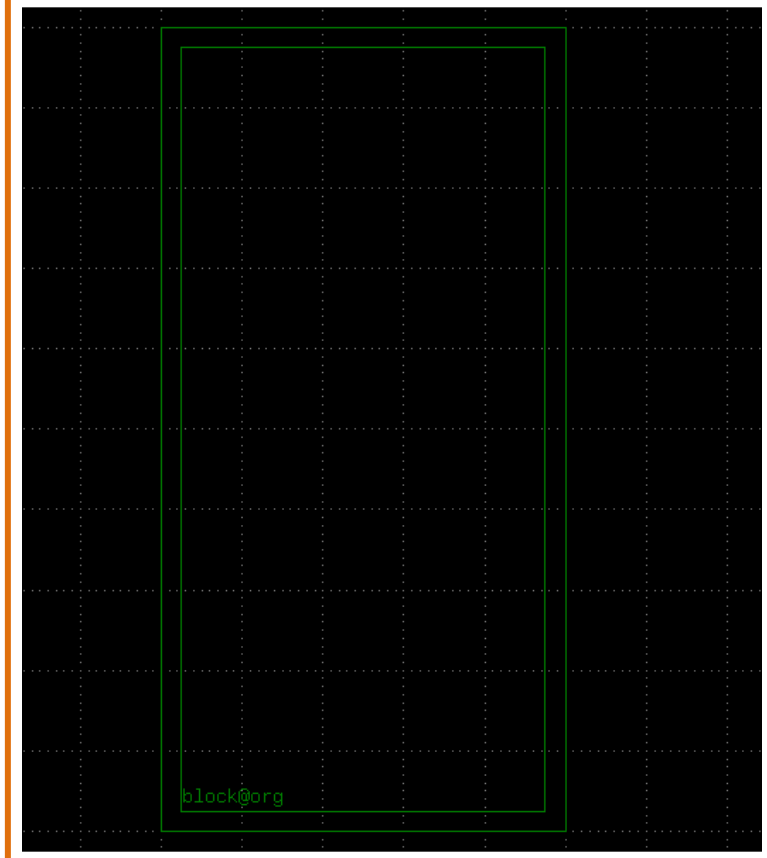


Financiado por la Unión Europea NextGenerationEU
GOBIERNO DE ESPAÑA MINISTERIO PARA LA TRANSFORMACIÓN DIGITAL Y DE LA FUNCIÓN PÚBLICA
Plan de Recuperación, Transformación y Resiliencia
España | digital 2026
PERTE Chip
UNIVERSITAT POLITÈCNICA DE VALÈNCIA
TSI-069100-2023-0002

Learning outcome #4 – Notebook section 4. Exercises – Part b) Creating die

Code for …
- Create a die W = 5 mm x L = 10 mm

```
gf.clear_cache()

@gf.cell
def die_WL(dieL = 10000, dieW = 5000, border = 250, layer_box = "FLOORPLAN"):
    # Die specifications (Chip)
    box = gf.Component()
    obox = box.add_ref(gf.components.rectangle(size=(dieW,dieL),layer=layer_box))
    ibox = box.add_ref(gf.components.rectangle(size=(dieW-border*2,dieL-border*2),layer=layer_box)).dmovex(border).dmovey(border)
    box = gf.boolean(A=obox, B=ibox, operation="A-B", layer=layer_box)
    # Adding ports to a component
    box.add_port(name="block@org", center=[border,border], width=1, orientation=0, layer=layer_box)
    box.draw_ports()

    return box

# 2. We instantiate references of each cell. We can also name it individually to avoid problems
wafer = gf.Component()
dieW = 5000

c1 = wafer.add_ref(die_WL(dieW = dieW))

wafer.show()
wafer.plot()
```



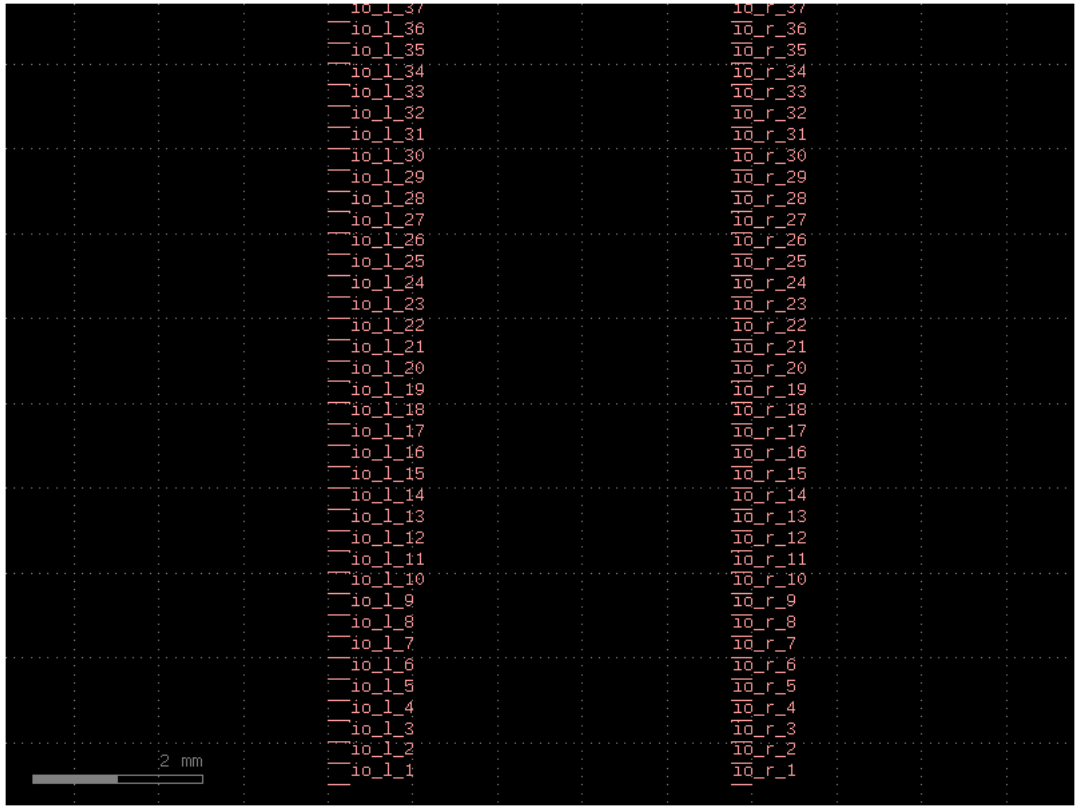
Financiado por la Unión Europea NextGenerationEU
TSI-069100-2023-0002

Learning outcome #4 – Notebook section 4. Exercises – Part b) Creating die

Code for …

- Create an array of I/Os spaced at 250 µm (as much I/Os as the width allows)

```
port_preview = gf.Component()
n=37
y_start = 500
for i in range(n):
    y_pos = y_start + i * sp

    in_arr = port_preview.add_ref(
        gf.components.straight(length=border, cross_section=xs)
    ).dmovex(0).dmovey(y_pos)
    port_preview.add_port(name=f"io_l_{i+1}", port=in_arr['o2'])

    out_arr = port_preview.add_ref(
        gf.components.straight(length=border, cross_section=xs)
    ).drotate(180).dmovex(5000).dmovey(y_pos)
    port_preview.add_port(name=f"io_r_{i+1}", port=out_arr['o2'])

port_preview.draw_ports()
port_preview.plot()
```



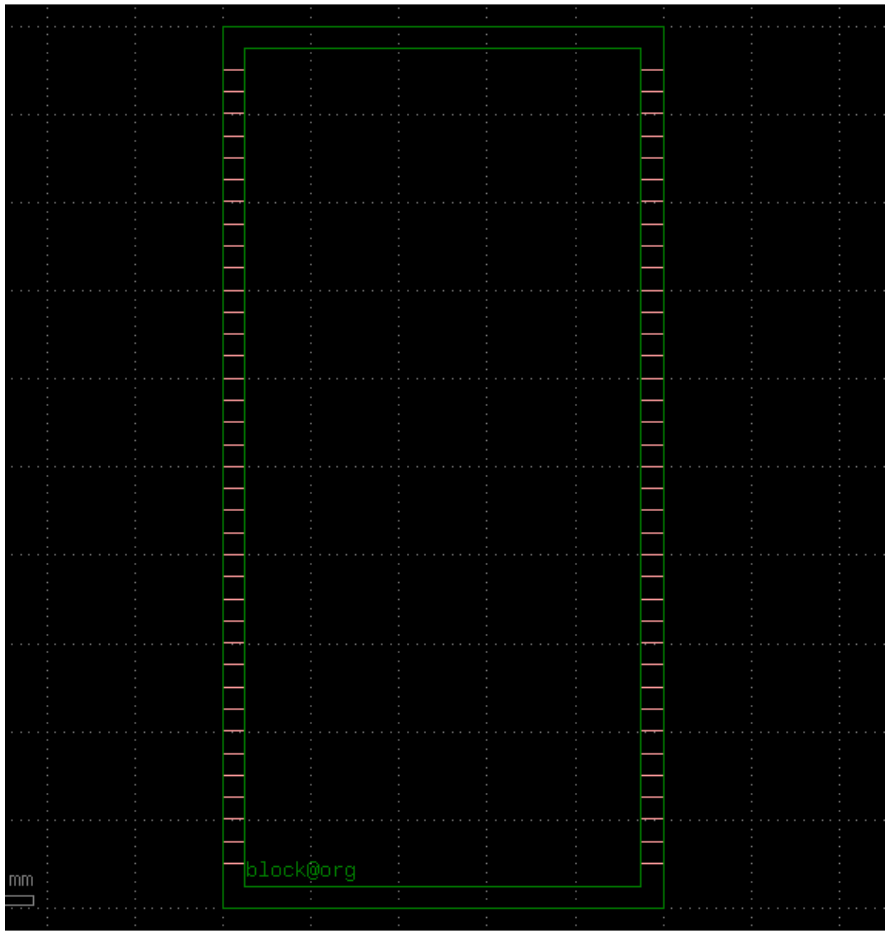
Financiado por la Unión Europea NextGenerationEU
Plan de Recuperación, Transformación y Resiliencia
España | digital 2026
PERTE Chip
UNIVERSITAT POLITÈCNICA DE VALÈNCIA
TSI-069100-2023-0002

Learning outcome #4 – Notebook section 4. Exercises – Part b) Creating die

Code for …

- Create a cell component of this die, with I/Os accessible to connect

```
gf.clear_cache()

sp = 250
border = 250
n=37

@gf.cell
def die_WL(dieL = 10000, dieW = 5000, border = 250, layer_box = "FLOORPLAN"):
    # Die specifications (Chip)
    box = gf.Component()
    obox = box.add_ref(gf.components.rectangle(size=(dieW,dieL),layer=layer_box))
    ibox = box.add_ref(gf.components.rectangle(size=(dieW-border*2,dieL-border*2),layer=layer_box)).dmovex(border).dmovey(border)
    box = gf.boolean(A=obox, B=ibox, operation="A-B", layer=layer_box)
    # Adding ports to a component
    box.add_port(name="block@org", center=[border,border], width=1, orientation=0, layer=layer_box)
    box.draw_ports()

    y_start = 500
    for i in range(n):
        y_pos = y_start + i * sp

        # Entrada izquierda: o2 apunta hacia el centro
        in_arr = box.add_ref(
            gf.components.straight(length=border, cross_section=xs)
        ).dmovex(0).dmovey(y_pos)
        box.add_port(name=f"io_l_{i+1}", port=in_arr['o2'])

        # Salida derecha: rotada 180°, o2 apunta hacia el centro
        out_arr = box.add_ref(
            gf.components.straight(length=border, cross_section=xs)
        ).drotate(180).dmovex(dieW).dmovey(y_pos)
        box.add_port(name=f"io_r_{i+1}", port=out_arr['o2'])


    return box
#gf.clear_cache()
# 2. We instantiate references of each cell. We can also name it individually to avoid problems
wafer = gf.Component()
dieW = 5000

c1 = wafer.add_ref(die_WL(dieW = dieW))

wafer.show()
wafer.plot()
```

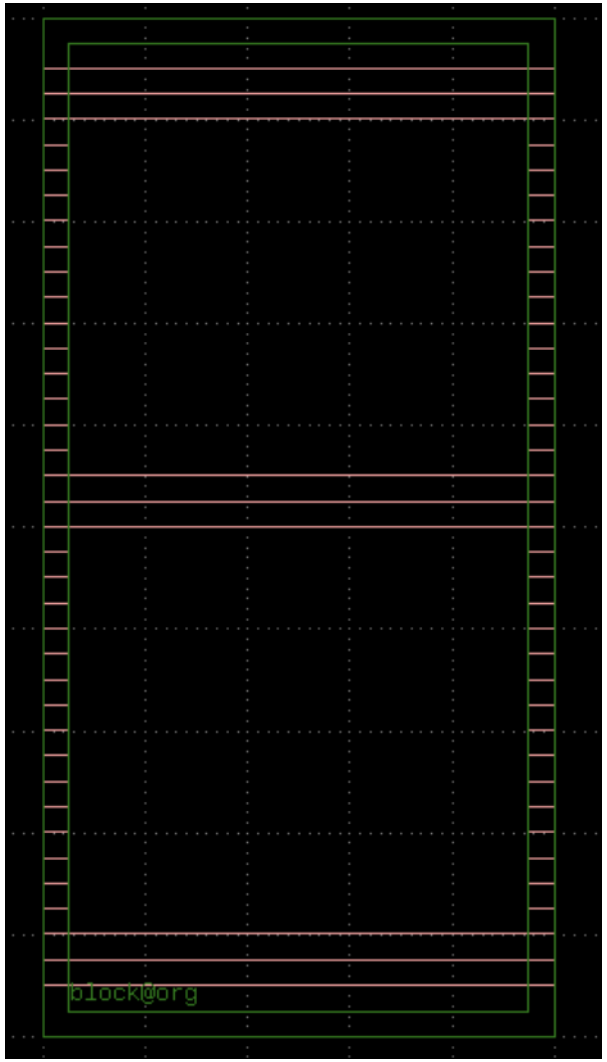


Financiado por la Unión Europea NextGenerationEU
GOBIERNO DE ESPAÑA MINISTERIO PARA LA TRANSFORMACIÓN DIGITAL Y DE LA FUNCIÓN PÚBLICA
Plan de Recuperación, Transformación y Resiliencia
España | digital 2026
PERTE Chip
UNIVERSITAT POLITÈCNICA DE VALÈNCIA
TSI-069100-2023-0002

Learning outcome #4 – Notebook section 4. Exercises – Part c) Floorplaning and die layout

Code for …
- Make an instance of the die as host component for your layout
- Add 3 sets of 3 straight waveguides, from left to right of the die, top, middle and bottom of the die

```
layout = gf.Component()
die_WG = layout.add_ref(die_WL(dieL = 10000, dieW = 5000, border = 250, layer_box = "FLOORPLAN"))

rows = {
    "bottom": [1,  2,  3 ],
    "middle": [19, 20, 21],
    "top"   : [35, 36, 37],
}

def add_straight_row(parent, io_indices):
    for i in io_indices:
        gf.routing.route_single(
            parent,
            port1         = die_WG.ports[f'io_l_{i}'],
            port2         = die_WG.ports[f'io_r_{i}'],
            cross_section = xs,
            allow_width_mismatch=True,
        )

add_straight_row(layout, rows["bottom"])
add_straight_row(layout, rows["middle"])
add_straight_row(layout, rows["top"])

layout.plot()
layout.show()
```



Financiado por la Unión Europea NextGenerationEU
TSI-069100-2023-0002

Learning outcome #4 – Notebook section 4. Exercises – Part c) Floorplaning and die

Code for …
- Add 2 of each of the components above, with different lengths
- Connect all your components to the I/Os

```
# Espiral 1
sp1 = layout.add_ref(spiral(length=1000))
sp1.dmovey(die_WG['io_l_4'].dy - sp1['o1'].dy)
sp1.dmovex(0.5*5000 - sp1.dxsize/2)


gf.routing.route_single(layout,port1=die_WG.ports['io_l_4'], port2=sp1.ports['o2'],
                        cross_section=xs, allow_width_mismatch=True)
gf.routing.route_single(layout,port1=sp1.ports['o1'], port2=die_WG.ports['io_r_4'],
                        cross_section=xs, allow_width_mismatch=True)

# Espiral 2
sp2 = layout.add_ref(spiral(length=2000))
sp2.dmovex(0.5*5000 - sp2.dxsize/2).dmovey(die_WG.ports['io_l_22'].dy - sp2.ports['o1'].dy)

gf.routing.route_single(layout, port1=die_WG.ports['io_l_22'], port2=sp2.ports['o2'],
                        cross_section=xs, allow_width_mismatch=True)
gf.routing.route_single(layout,port1=sp2.ports['o1'], port2=die_WG.ports['io_r_22'],
                        cross_section=xs, allow_width_mismatch=True)
```

```
# Ring heater1
r1_h = layout.add_ref(ring_heater_cell(radio=200.0))
r1_h.dmovex(0.5*5000 - r1_h.dxsize/2).dmovey(die_WG.ports['io_l_6'].dy - r1_h.ports['o1'].dy)

gf.routing.route_single(layout,port1=die_WG.ports['io_l_6'], port2=r1_h.ports['o2'],
                        cross_section=xs, allow_width_mismatch=True)
gf.routing.route_single(layout,port1=r1_h.ports['o2'], port2=die_WG.ports['io_r_6'],
                        cross_section=xs, allow_width_mismatch=True)

# Ring heater2
r2_h = layout.add_ref(ring_heater_cell(radio=100.0))
r2_h.dmovex(0.5*5000 - r2_h.dxsize/2).dmovey(die_WG.ports['io_l_24'].dy - r2_h.ports['o1'].dy)

gf.routing.route_single(layout, port1=die_WG.ports['io_l_24'], port2=r2_h.ports['o2'],
                        cross_section=xs, allow_width_mismatch=True)
gf.routing.route_single(layout,port1=r2_h.ports['o2'], port2=die_WG.ports['io_r_24'],
                        cross_section=xs, allow_width_mismatch=True)
```

```
# Ring 1
r1 = layout.add_ref(ring_resonator_cell(radio=200.0))
r1.dmovex(0.5*5000 - r1.dxsize/2).dmovey(die_WG.ports['io_l_8'].dy - r1.ports['o1'].dy)

gf.routing.route_single(layout,port1=die_WG.ports['io_l_8'], port2=r1.ports['o2'],
                        cross_section=xs, allow_width_mismatch=True)
gf.routing.route_single(layout,port1=r1.ports['o2'], port2=die_WG.ports['io_r_8'],
                        cross_section=xs, allow_width_mismatch=True)

# Ring 2
r2 = layout.add_ref(ring_resonator_cell(radio=100.0))
r2.dmovex(0.5*5000 - r2.dxsize/2).dmovey(die_WG.ports['io_l_26'].dy - r2.ports['o1'].dy)

gf.routing.route_single(layout, port1=die_WG.ports['io_l_26'], port2=r2.ports['o2'],
                        cross_section=xs, allow_width_mismatch=True)
gf.routing.route_single(layout,port1=r2.ports['o2'], port2=die_WG.ports['io_r_26'],
                        cross_section=xs, allow_width_mismatch=True)
```

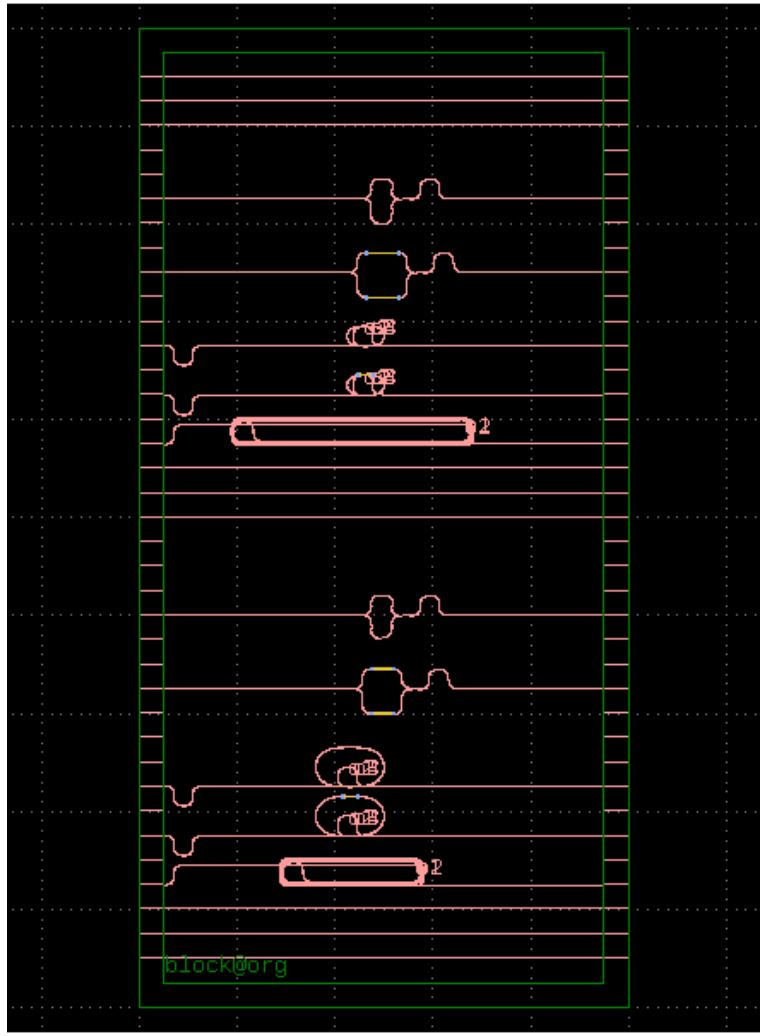
```
# mzi_heater 1
mzi_1 = layout.add_ref(mzi_heater_h(length_x=200))
mzi_1.dmovey(die_WG.ports['io_l_12'].dy - mzi_1['o1'].dy)
mzi_1.dmovex(0.5*5000 - mzi_1.dxsize/2)
gf.routing.route_single(layout, port1=die_WG.ports['io_l_12'], port2=mzi_1['o1'],
                        cross_section=xs, allow_width_mismatch=True)
gf.routing.route_single(layout, port1=mzi_1['o3'], port2=die_WG.ports['io_r_12'],
                        cross_section=xs, allow_width_mismatch=True)

# mzi_heater 2
mzi_2 = layout.add_ref(mzi_heater_h(length_x=300))
mzi_2.dmovey(die_WG.ports['io_l_29'].dy - mzi_2['o1'].dy)
mzi_2.dmovex(0.5*5000 - mzi_2.dxsize/2)
gf.routing.route_single(layout, port1=die_WG.ports['io_l_29'], port2=mzi_2['o1'],
                        cross_section=xs, allow_width_mismatch=True)
gf.routing.route_single(layout, port1=mzi_2['o3'], port2=die_WG.ports['io_r_29'],
                        cross_section=xs, allow_width_mismatch=True)
```


Financiado por la Unión Europea NextGenerationEU
GOBIERNO DE ESPAÑA MINISTERIO PARA LA TRANSFORMACIÓN DIGITAL Y DE LA FUNCIÓN PÚBLICA
Plan de Recuperación, Transformación y Resiliencia
España | digital 2026
PERTE Chip
UNIVERSITAT POLITÈCNICA DE VALÈNCIA
TSI-069100-2023-0002

Learning outcome #4 – Notebook section 4. Exercises – Part c) Floorplaning and die

Code for …
- Add 2 of each of the components above, with different lengths
- Connect all your components to the I/Os

```
# mzi_unbalanced 1
mzi_u_1 = layout.add_ref(mzi_unbalanced_2x2(delta_length=50.0))
mzi_u_1.dmovey(die_WG.ports['io_l_15'].dy - mzi_u_1['o1'].dy)
mzi_u_1.dmovex(0.5*5000 - mzi_u_1.dxsize/2)
gf.routing.route_single(layout, port1=die_WG.ports['io_l_15'], port2=mzi_u_1['o1'],
                        cross_section=xs, allow_width_mismatch=True)
gf.routing.route_single(layout, port1=mzi_u_1['o3'], port2=die_WG.ports['io_r_15'],
                        cross_section=xs, allow_width_mismatch=True)

# mzi_unbalanced 2
mzi_u_2 = layout.add_ref(mzi_unbalanced_2x2(delta_length=100.0))
mzi_u_2.dmovey(die_WG.ports['io_l_32'].dy - mzi_u_2['o1'].dy)
mzi_u_2.dmovex(0.5*5000 - mzi_u_2.dxsize/2)
gf.routing.route_single(layout, port1=die_WG.ports['io_l_32'], port2=mzi_u_2['o1'],
                        cross_section=xs, allow_width_mismatch=True)
gf.routing.route_single(layout, port1=mzi_u_2['o3'], port2=die_WG.ports['io_r_32'],
                        cross_section=xs, allow_width_mismatch=True)

layout.plot()
layout.show()
```



Financiado por la Unión Europea NextGenerationEU
TSI-069100-2023-0002

PIC-UPV
PERTE CHIP CHAIR

Thank you

Financiado por la Unión Europea NextGenerationEU
GOBIERNO DE ESPAÑA MINISTERIO PARA LA TRANSFORMACIÓN DIGITAL Y DE LA FUNCIÓN PÚBLICA
Plan de Recuperación, Transformación y Resiliencia
España | digital 2026
PERTE Chip
UNIVERSITAT POLITÈCNICA DE VALÈNCIA
TSI-069100-2023-0002